

Introduction to Machine Learning

Week 6 - Laplace Approximation of Posterior

Spring 2025

Instructor: Dr. Patrick Skeba

How can we use a MVN to help us?

- Denote the parameters of interest as θ .
- For the weight example, $\theta = \{\mu, \sigma\}$.
- Assume we can find the posterior mode, or MAP, denote as $\hat{\theta}$.

How can we use a MVN to help us?

- A *second-order Taylor series expansion* of the log-posterior **around the posterior mode**.

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{g}\big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H}\big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

How can we use a MVN to help us?

- A *second-order Taylor series expansion* of the log-posterior **around the posterior mode**.

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{g}\big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H}\big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

Gradient vector of the log-posterior with respect to each parameter of interest, evaluated at the posterior mode.

For the weight example: $\mathbf{g} = \left\{ \frac{\partial}{\partial \mu} (\log[p(\boldsymbol{\theta}|\mathbf{x})]), \frac{\partial}{\partial \sigma} (\log[p(\boldsymbol{\theta}|\mathbf{x})]) \right\}$

How can we use a MVN to help us?

- A *second-order Taylor series expansion* of the log-posterior **around the posterior mode**.

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \boxed{\mathbf{g}|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}} + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H}|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

Gradient vector of the log-posterior with respect to each parameter of interest, evaluated at the posterior mode.

At $\boldsymbol{\theta} = \hat{\boldsymbol{\theta}}$ the gradient, $\mathbf{g}$, by definition, is equal to...

How can we use a MVN to help us?

- A *second-order Taylor series expansion* of the log-posterior **around the posterior mode**.

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \boxed{\mathbf{g}|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}} + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H}|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

Gradient vector of the log-posterior with respect to each parameter of interest, evaluated at the posterior mode.

At $\boldsymbol{\theta} = \hat{\boldsymbol{\theta}}$ the gradient, $\mathbf{g}$, by definition, is equal to...**0!!!!!!!**

The second-order Taylor series expansion simplifies to:

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H} \Big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

The second-order Taylor series expansion simplifies to:

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H} \Big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

The **Hessian matrix** of the log-posterior with respect to the pair-wise parameter combinations evaluated at the posterior mode.

The Hessian matrix is a matrix of second derivatives and represents the **local curvature** of the surface.

The second-order Taylor series expansion simplifies to:

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H} \Big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

For the weight example, the Hessian matrix is a 2×2 matrix of second derivatives:

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2}{\partial \mu^2} (\log[p(\boldsymbol{\theta}|\mathbf{x})]) & \frac{\partial^2}{\partial \mu \partial \sigma} (\log[p(\boldsymbol{\theta}|\mathbf{x})]) \\ \frac{\partial^2}{\partial \sigma \partial \mu} (\log[p(\boldsymbol{\theta}|\mathbf{x})]) & \frac{\partial^2}{\partial \sigma^2} (\log[p(\boldsymbol{\theta}|\mathbf{x})]) \end{bmatrix}$$

What does this expression remind us of...?

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H} \Big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

What does this expression remind us of...?

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H} \Big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

Consider the log of a general MVN density is:

$$\log[\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}] = -\frac{D}{2} \log[2\pi] - \frac{1}{2} \log[|\boldsymbol{\Sigma}|] - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})$$

What does this expression remind us of...?

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \mathbf{H} \Big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

Focus on the quadratic term:

$$\log[\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}] \propto -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})$$

What does this expression remind us of...?

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + \frac{1}{2} (\boldsymbol{\theta} - \boxed{\hat{\boldsymbol{\theta}}})^T \mathbf{H} \Big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}} (\boldsymbol{\theta} - \boxed{\hat{\boldsymbol{\theta}}})$$

Focus on the quadratic term:

A vector of mean values!

$$\log[\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}] \propto -\frac{1}{2} (\mathbf{x} - \boxed{\boldsymbol{\mu}})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boxed{\boldsymbol{\mu}})$$

What does this expression remind us of...?

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \boxed{\mathbf{H}|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

Focus on the quadratic term:

Negative inverse of the
covariance matrix!

$$\log[\mathbf{x} | \boldsymbol{\mu}, \boldsymbol{\Sigma}] \propto -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \boxed{\boldsymbol{\Sigma}^{-1}} (\mathbf{x} - \boldsymbol{\mu})$$

What does this expression remind us of...?

$$\log[p(\boldsymbol{\theta}|\mathbf{x})] \approx \log[p(\hat{\boldsymbol{\theta}}|\mathbf{x})] + \frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^T \boxed{\mathbf{H} \Big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$$

Negative inverse of the
covariance matrix!

The negative of the Hessian matrix has a special
name...the **observed information matrix**.

The Laplace approximation

$$p(\boldsymbol{\theta}|\mathbf{x}) \approx \mathcal{N}\left(\hat{\boldsymbol{\theta}}, \left[-\mathbf{H}\big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}\right]^{-1}\right)$$

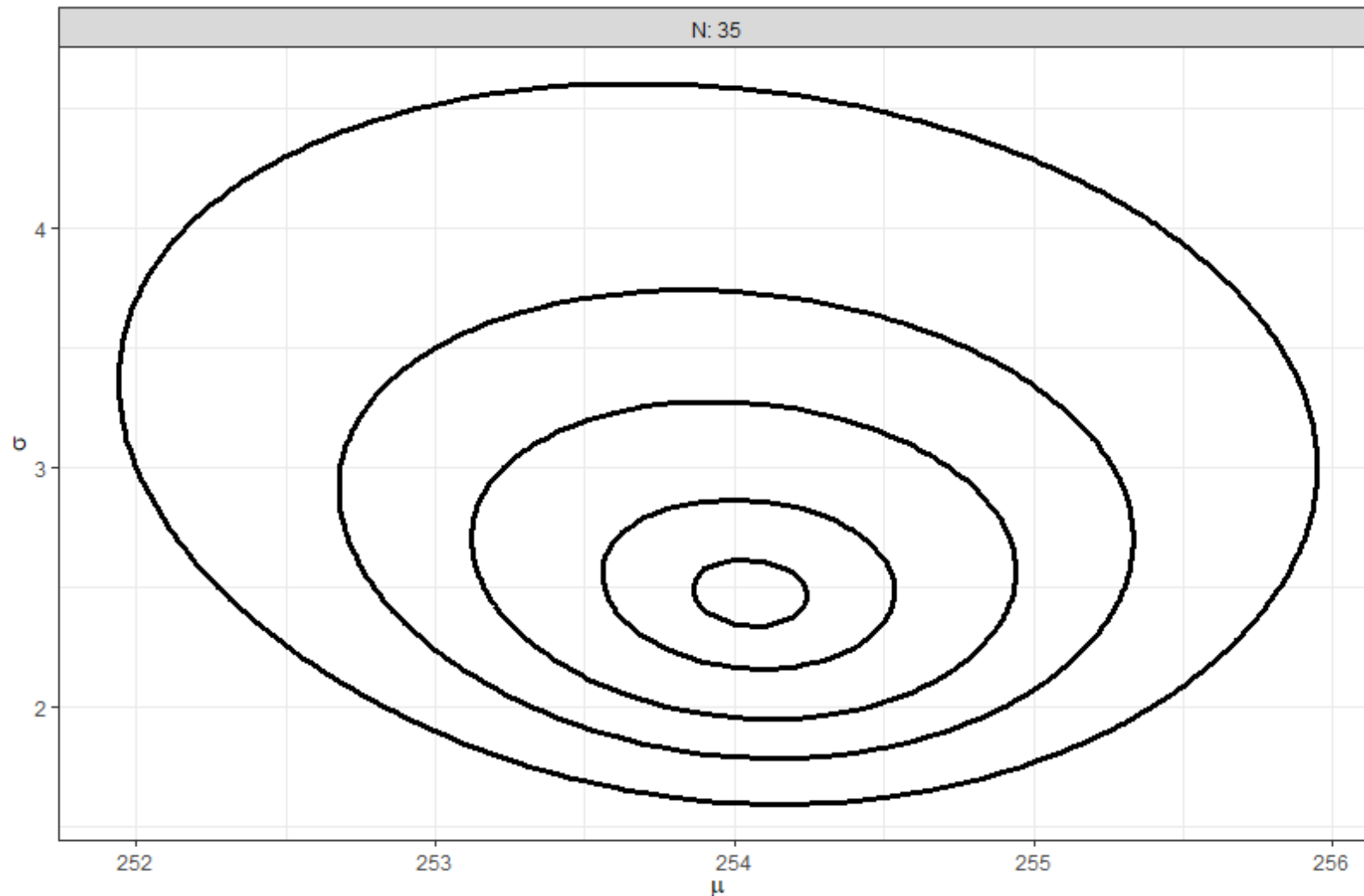
Let's apply the Laplace Approximation to the weight example

- The un-normalized log-posterior is:

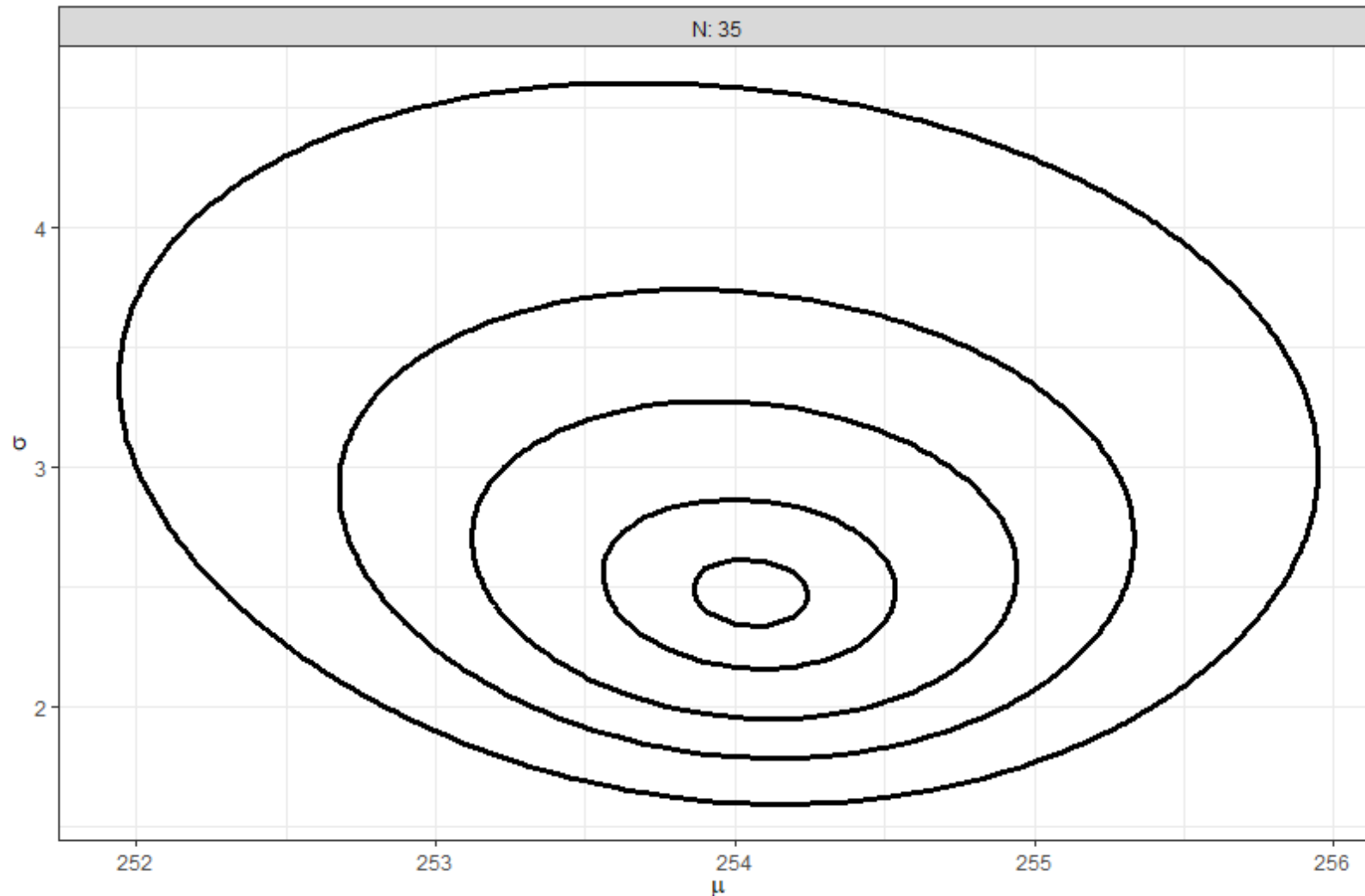
$$\log[p(\mu, \sigma \mid \mathbf{x})] \propto \sum_{n=1}^N (\log[\text{normal}(x_n \mid \mu, \sigma)]) + \log[\text{normal}(\mu \mid \mu_0, \tau_0)] + \log[\text{uniform}(\sigma \mid l, u)]$$

- With prior specification: $\mu_0 = 250$, $\tau_0 = 2$, $l = 0.5$, $u = 5.5$
- We will use 35 observations.

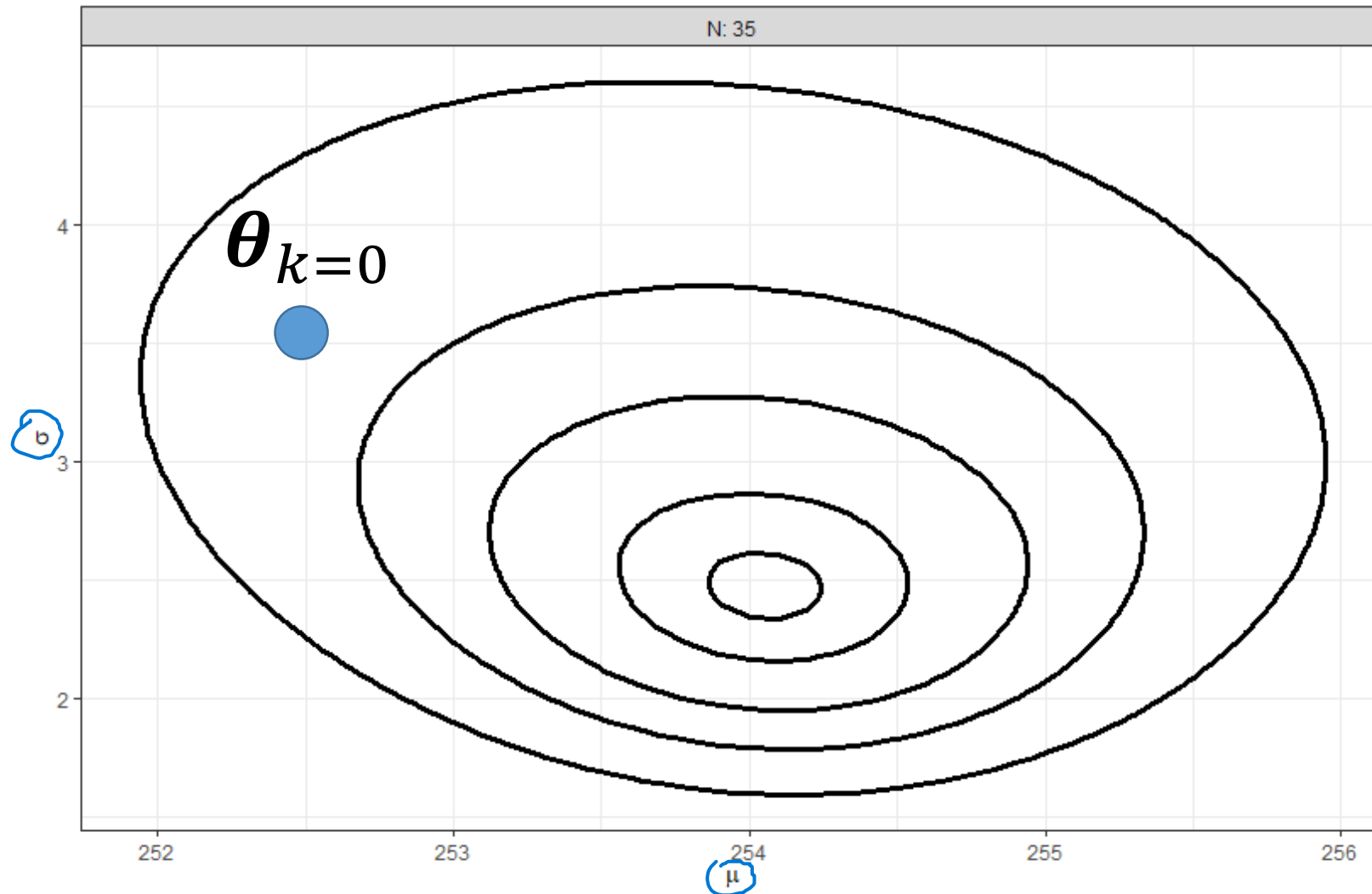
The log-posterior surface for $N = 35$



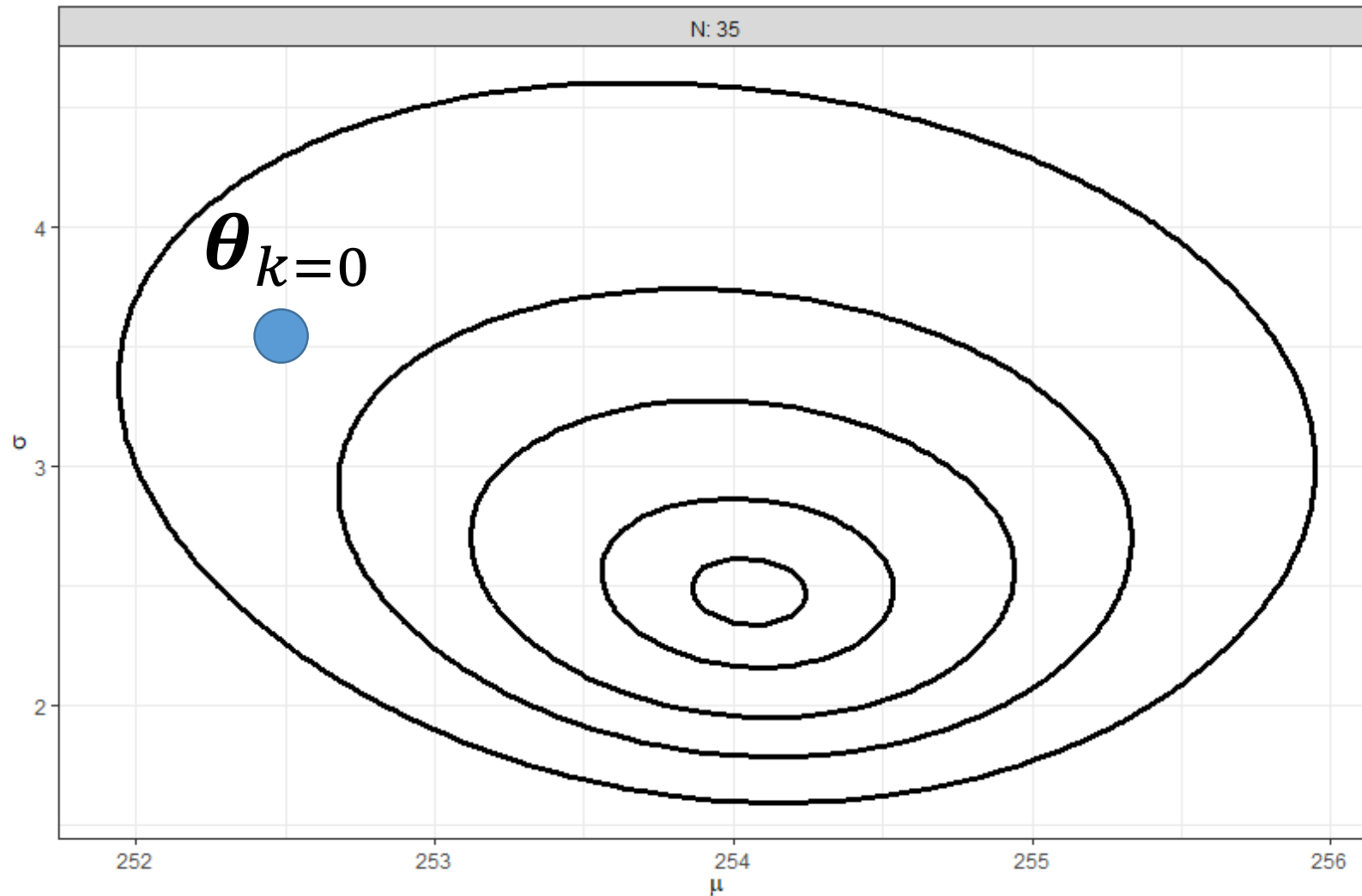
The **first step** in the Laplace Approximation is to find the MAP (the mode). How do we do that?



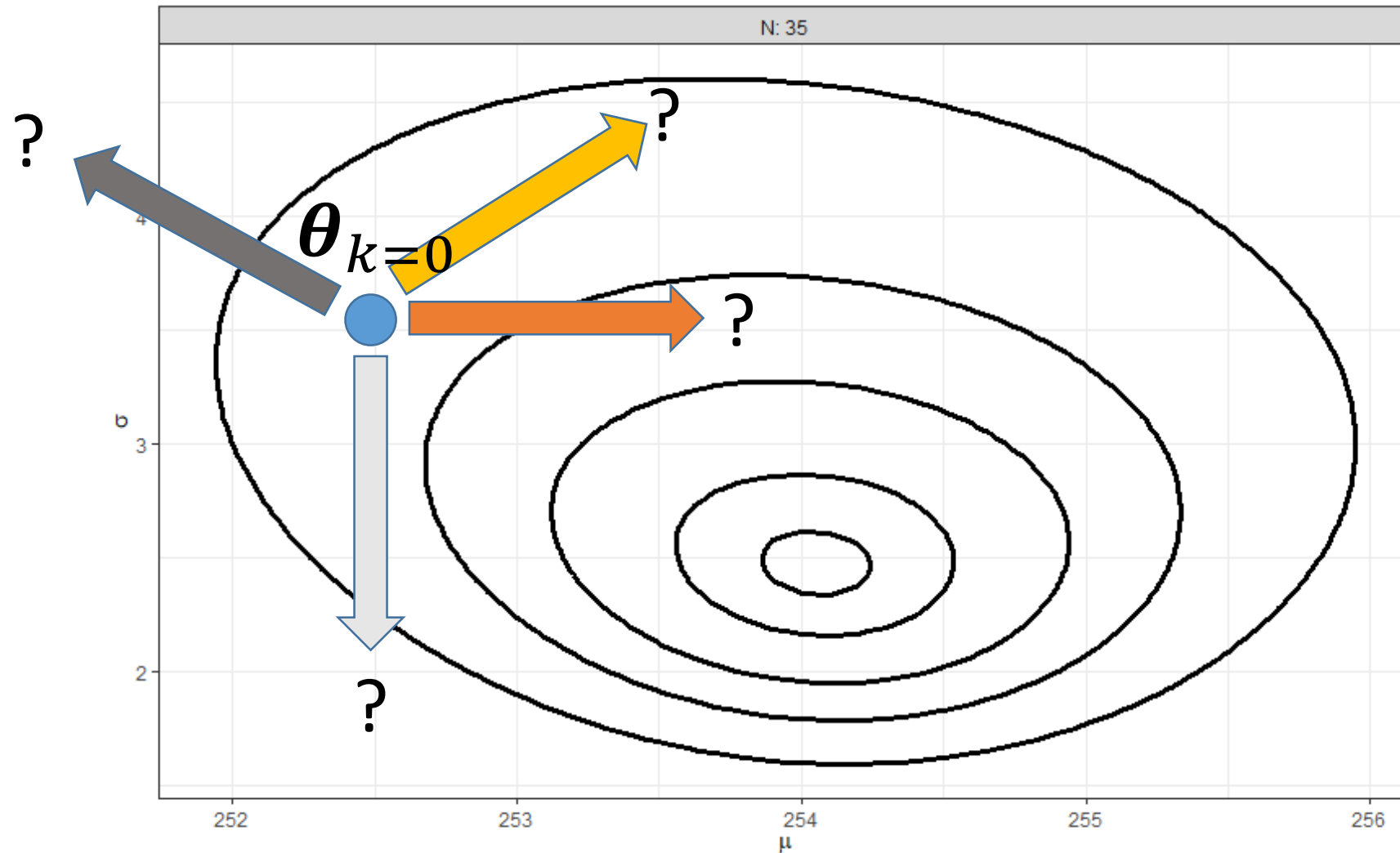
Start from an initial guess



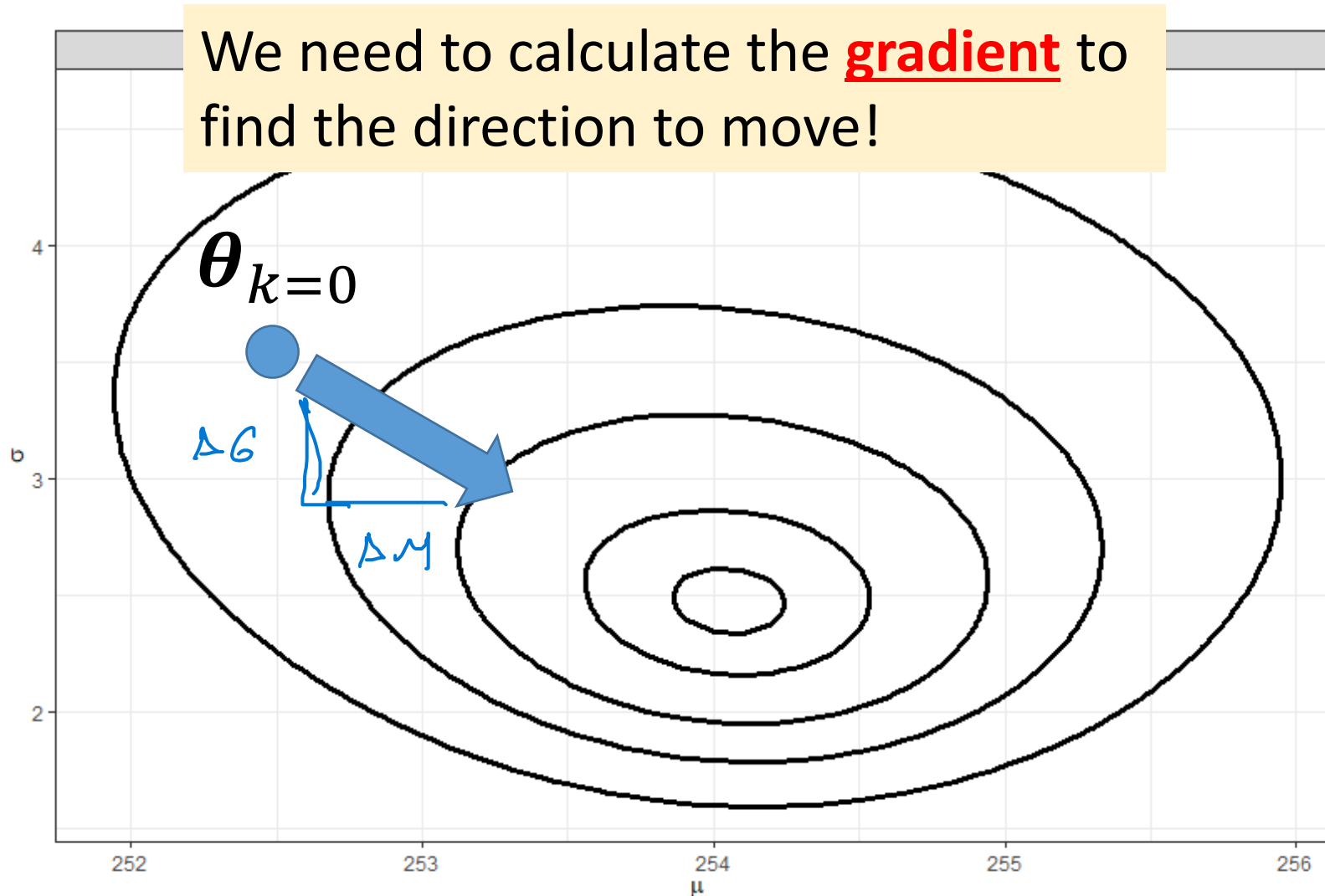
How do we update our guess?



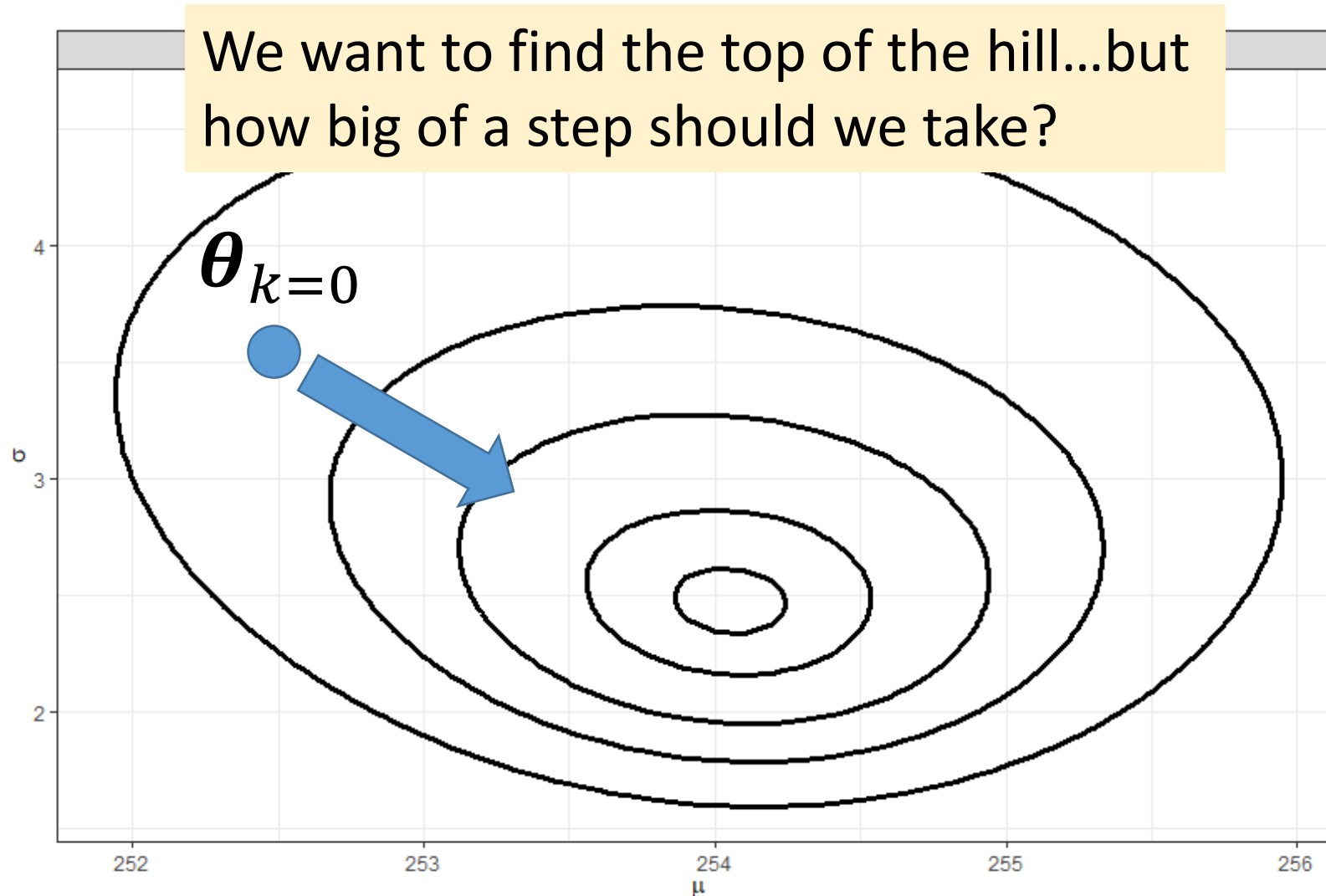
How do we update our guess?



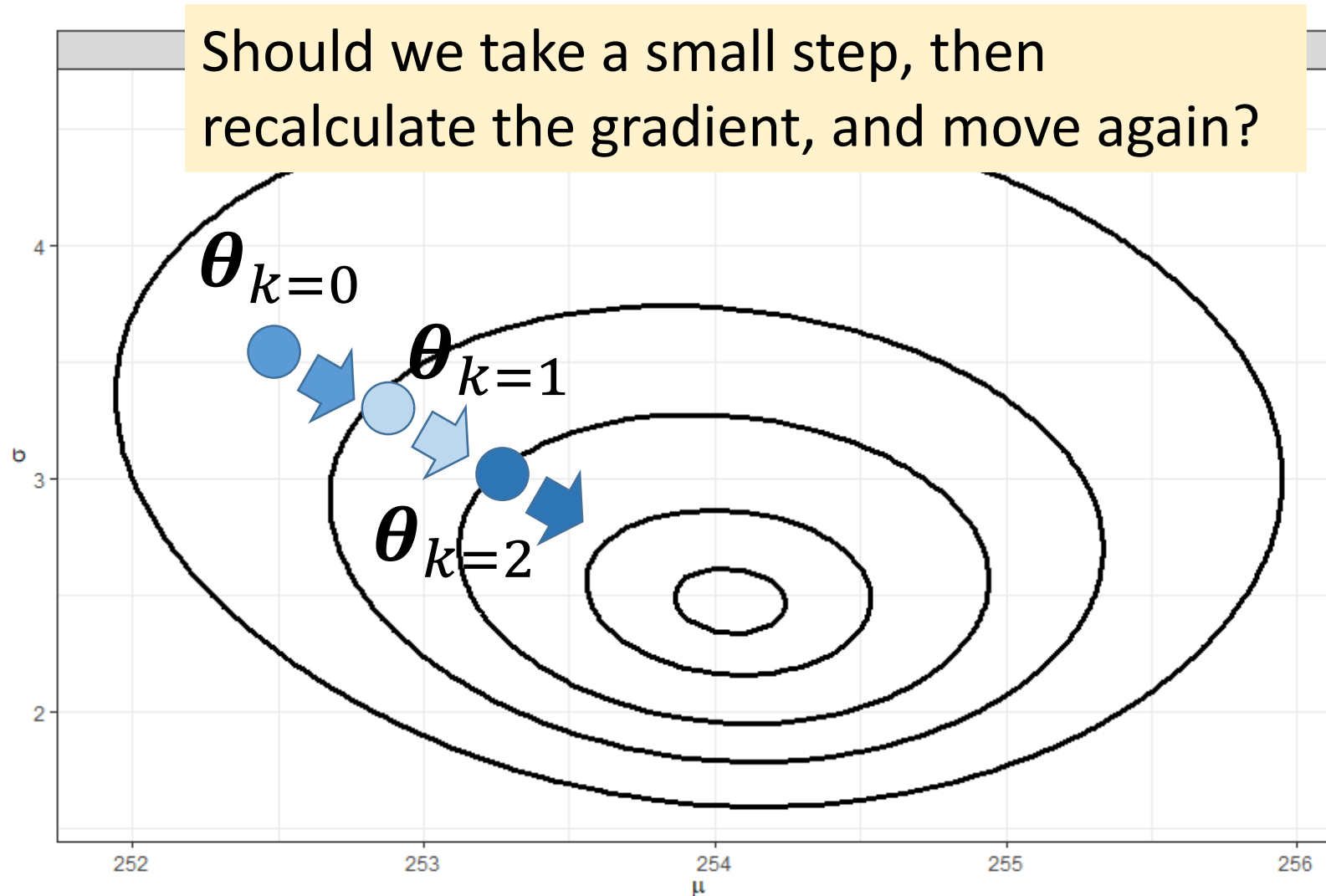
Climb the hill! Find the direction toward the “peak” and move that way!



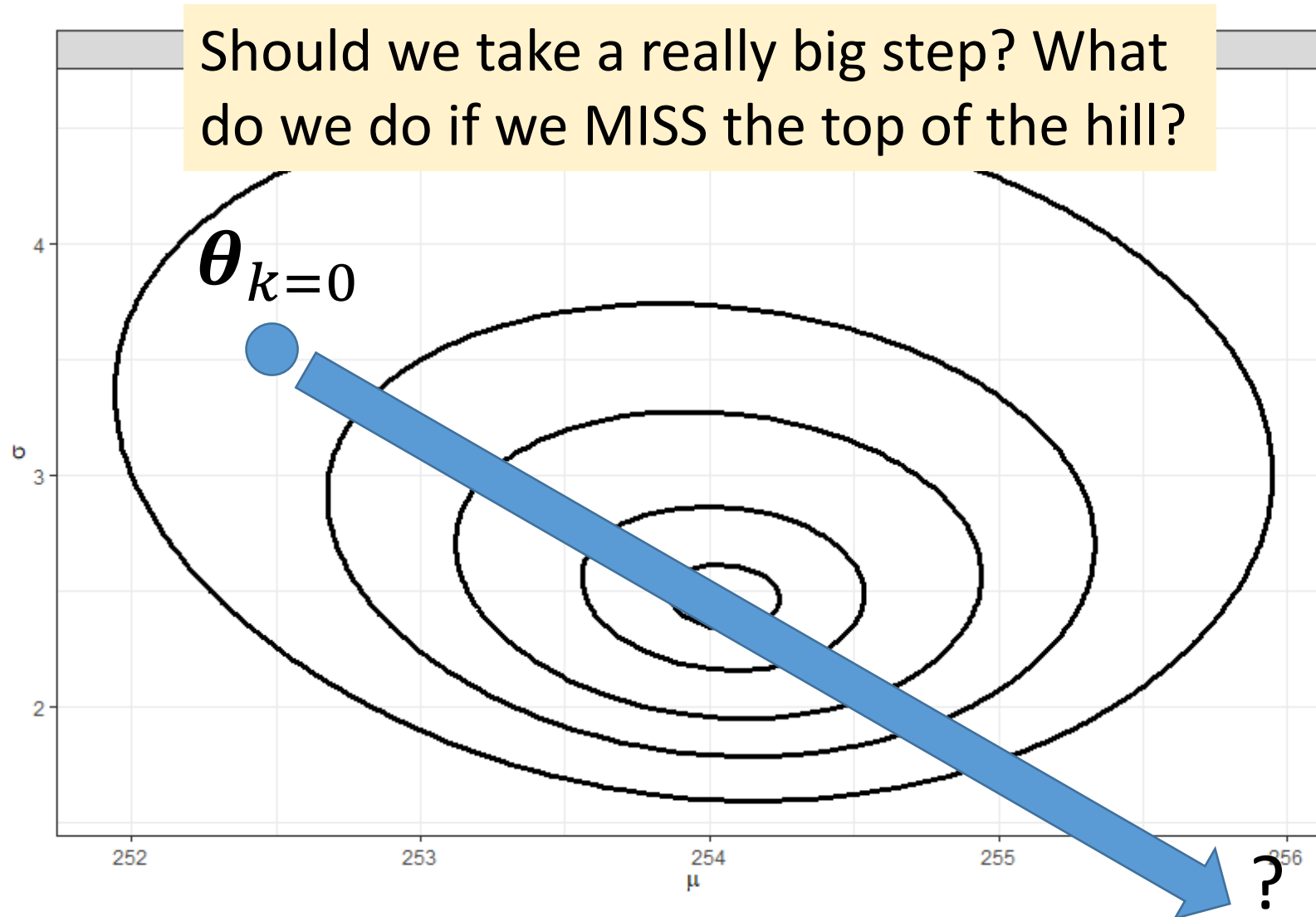
How far should we move in the direction defined by the gradient?



How far should we move in the direction defined by the gradient?



How far should we move in the direction defined by the gradient?



How far should we move in the direction defined by the gradient?

- Numerous algorithms exist for selecting the path or search length.
- When cast as a minimization problem, we need to find the **descent path**.
- In general, I'll say we need to find the STEP SIZE, $\Delta\theta$
- For the k -th iteration or step, the $k + 1$ (new) value is:

$$\theta_{k+1} = \theta_k + \gamma_k(\Delta\theta_k)$$

$\gamma_k \in (0,1)$ is a multiplier
which reduces the step size.

Gradient Ascent (maximize) or Gradient Descent (minimize)

- Move in the direction specified by the gradient:

$$\Delta \theta_k = - \mathbf{g} \Big|_k \quad \text{For a minimization problem}$$

- The $k + 1$ update is just the user specified fraction of the gradient:

$$\theta_{k+1} = \theta_k - \gamma \mathbf{g} \Big|_k$$

γ is the **learning rate**. Smaller values require more steps!

Newton-Raphson method – account for curvature

- We can use higher order derivatives to understand the behavior of the surface in more detail than just considering the slope.
- Newton-Raphson method sets the search length based on the Hessian!

$$\Delta \theta_k = - \left[\mathbf{H} \Big|_k \right]^{-1} \mathbf{g} \Big|_k$$

Hessian: matrix of second derivatives Gradient: vector of partial first derivatives

For a minimization problem

The curvature allows the step size to change and adapt as the surface changes!

Newton-Raphson method – account for curvature

- For the k -th iteration or step, the $k + 1$ (new) value is:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma_k \left[\mathbf{H} \Big|_k \right]^{-1} \mathbf{g} \Big|_k$$

For a minimization problem

$\gamma_k \in (0,1)$ is a multiplier which reduces the step size.

$\gamma_k = 1$ is a “Full Newton” step.

Fundamentally, the curvature controls how far we walk.

Newton-Raphson method – account for curvature

- For the k -th iteration or step, the $k + 1$ (new) value is:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma_k \left[\mathbf{H} \Big|_k \right]^{-1} \mathbf{g} \Big|_k$$

For a minimization problem

$\gamma_k \in (0,1)$ is a multiplier which reduces the step size.

$\gamma_k = 1$ is a “Full Newton” step.

How can we turn a
MAXIMIZATION problem
into a MINIMIZATION
problem???

Fundamentally, the curvature controls how far we walk.

Newton-Raphson method – account for curvature

- For the k -th iteration or step, the $k + 1$ (new) value is:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma_k \left[\mathbf{H} \Big|_k \right]^{-1} \mathbf{g} \Big|_k$$

For a minimization problem

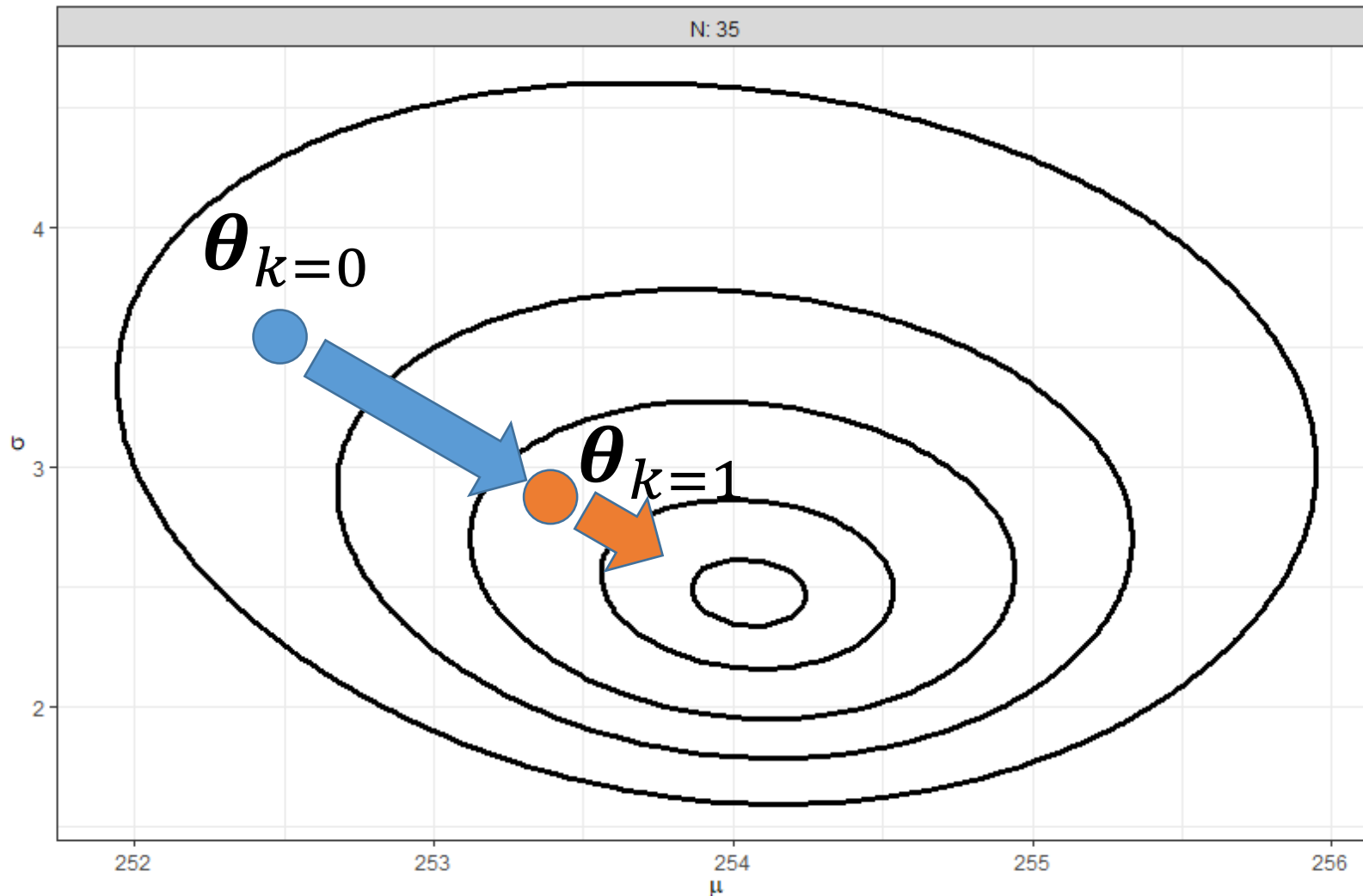
$\gamma_k \in (0,1)$ is a multiplier which reduces the step size.

MULTIPLY BY -1

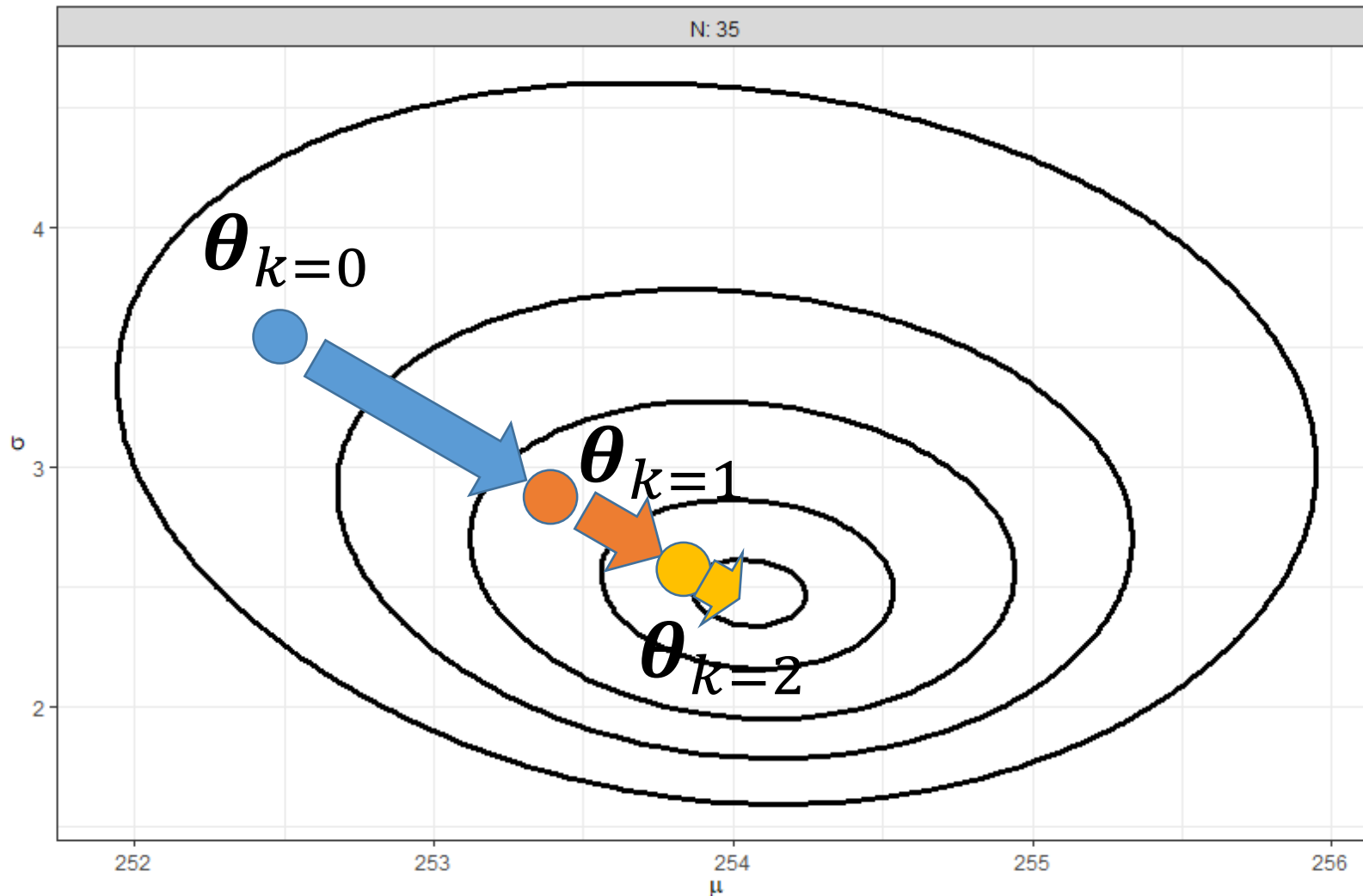
$\gamma_k = 1$ is a “Full Newton” step.

Fundamentally, the curvature controls how far we walk.

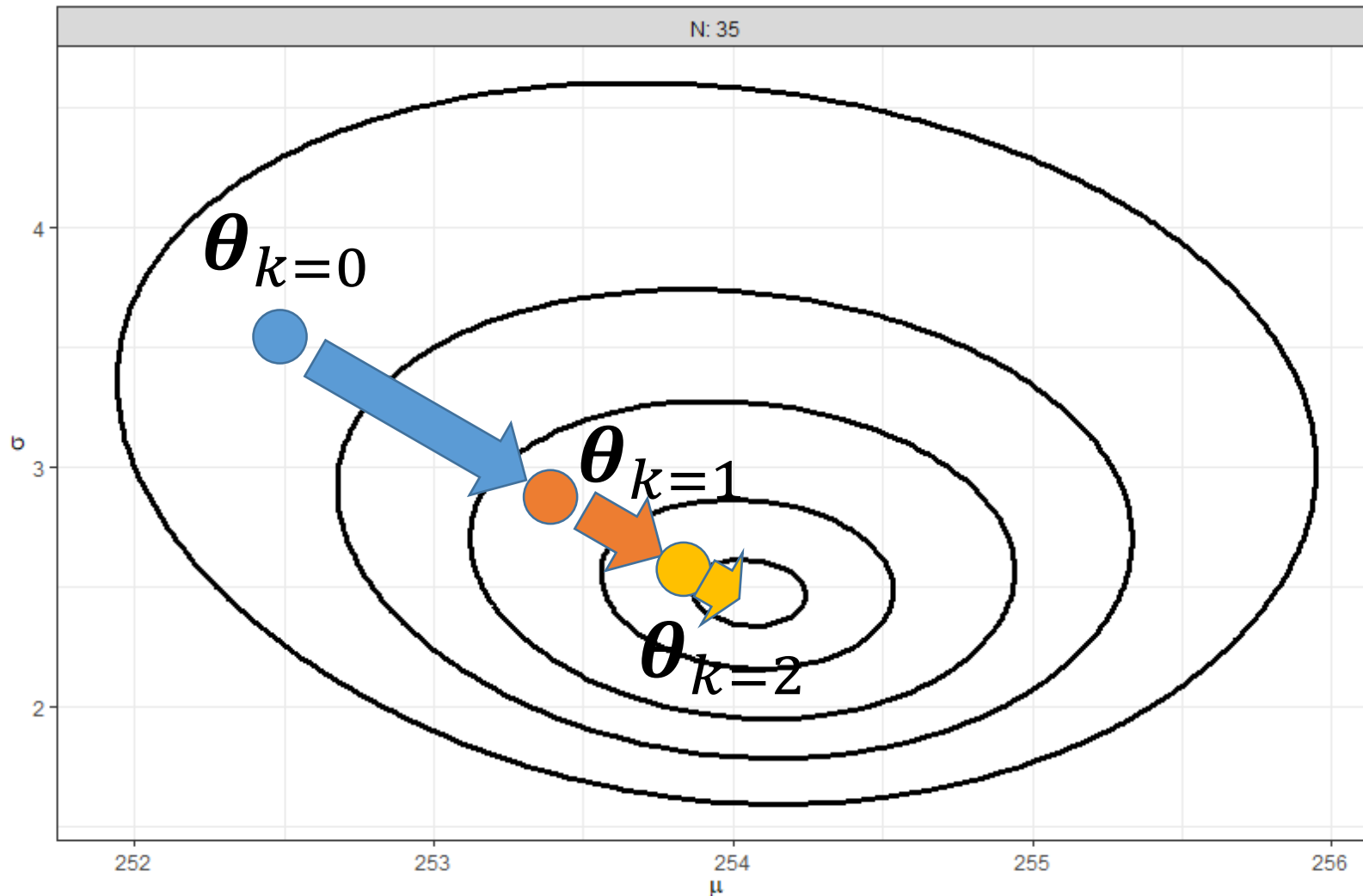
After we have made a move...we repeat the process until convergence!



After we have made a move...we repeat the process until convergence!

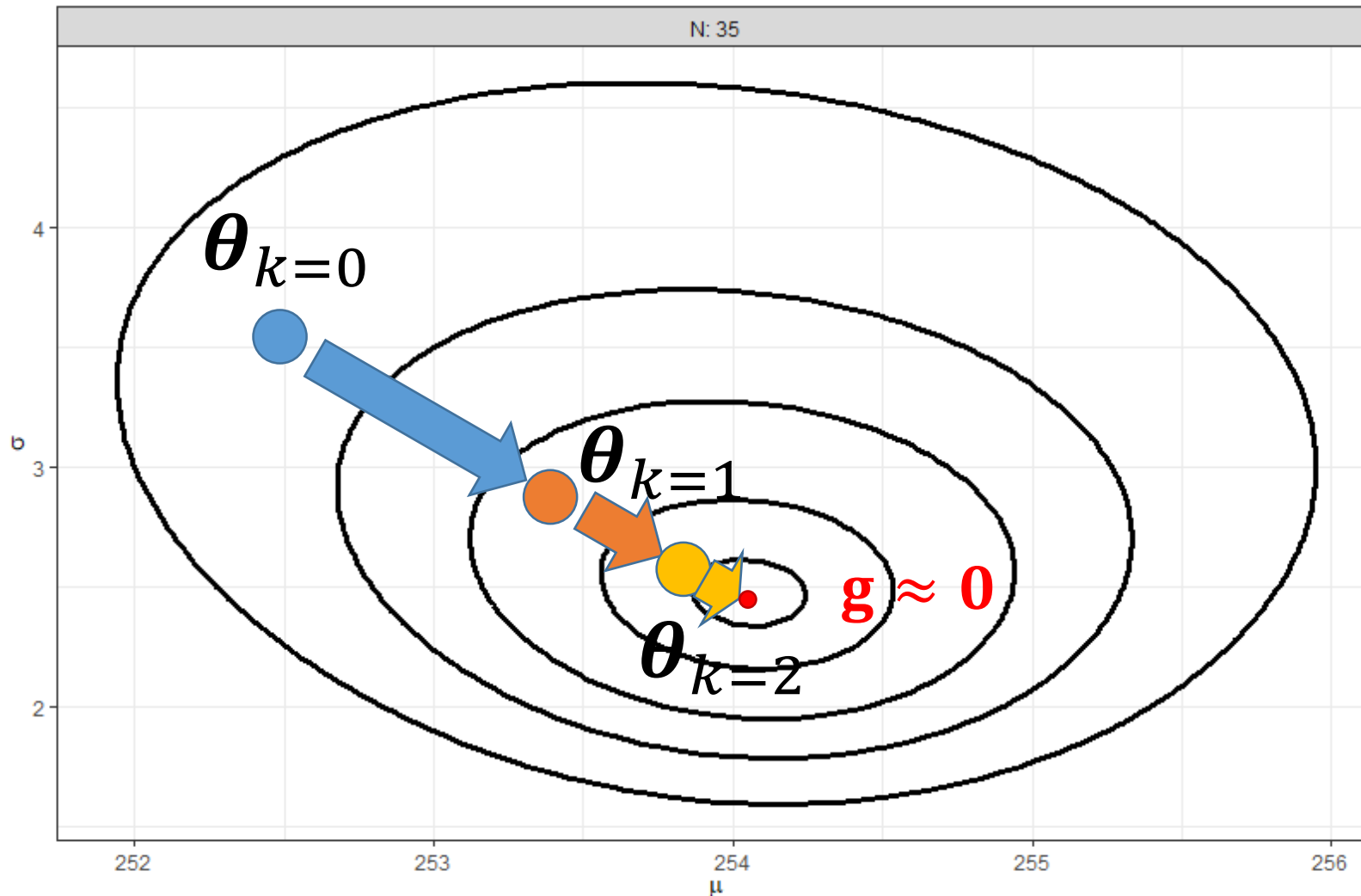


What does convergence mean?



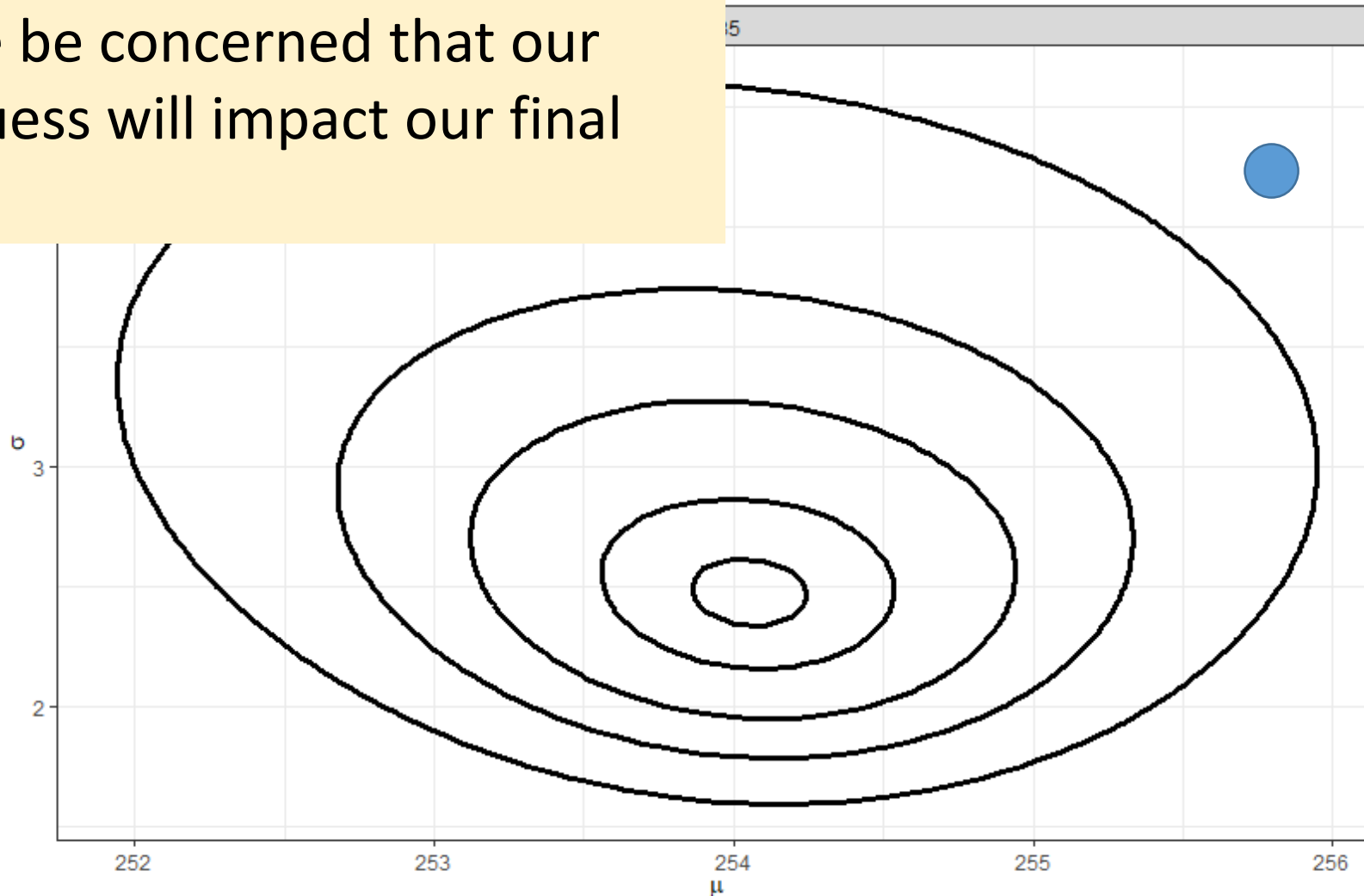
What does convergence mean?

Converge means the gradient is zero, $\mathbf{g} = \mathbf{0}$!



What if we started from somewhere else?

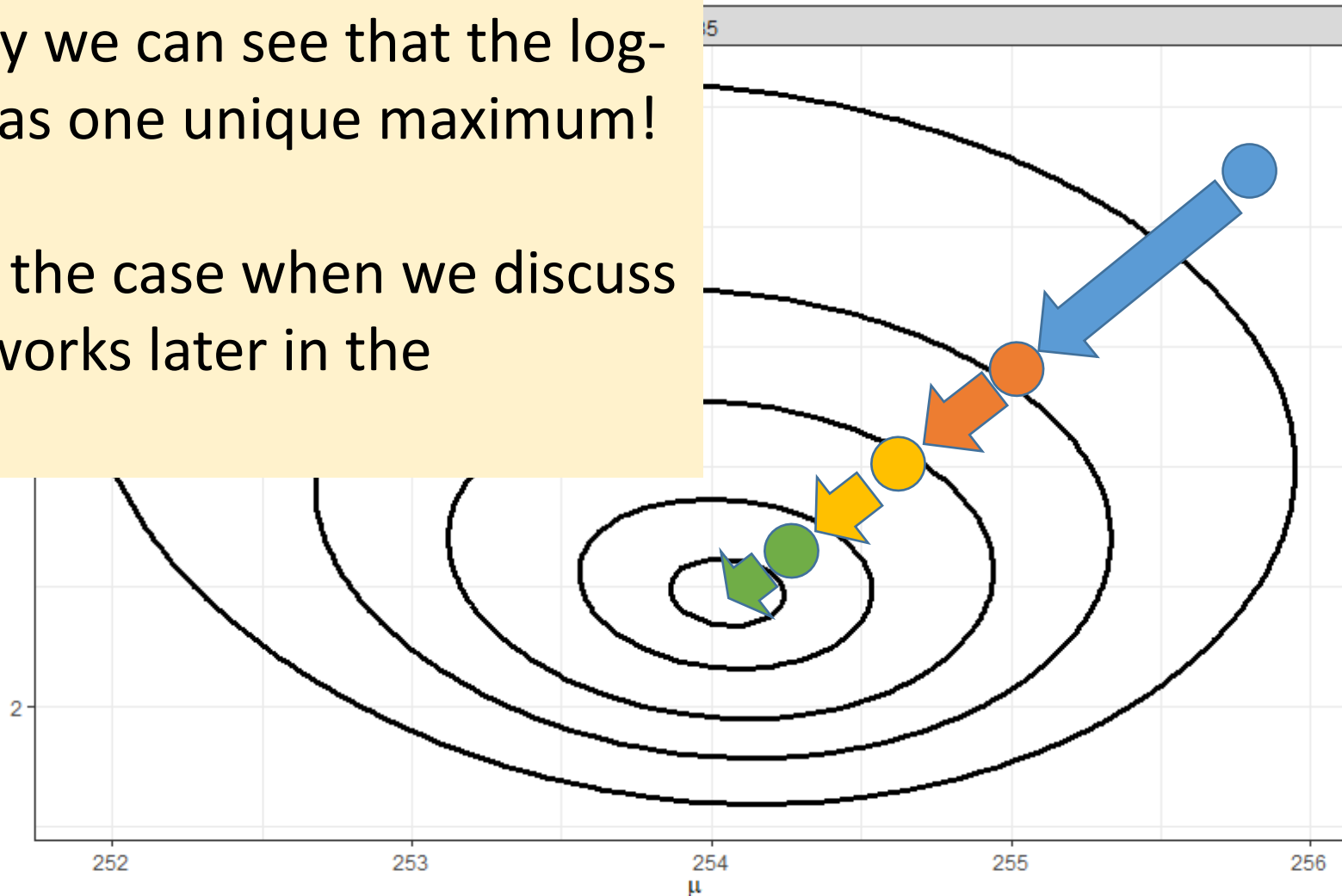
Should we be concerned that our starting guess will impact our final result?



What if we started from somewhere else?

NO! Visually we can see that the log-posterior has one unique maximum!

Will not be the case when we discuss neural networks later in the semester...



With an iterative gradient based optimization scheme we are constantly calculating derivatives!

- Need the gradient to define the direction.
- For a Newton method, we need the Hessian to decide how far to move.
- That's a lot of differentiation!

You do NOT need to program the optimization routine...

- We will use the `optim()` function in R to find the posterior mode.
- There are many different optimization algorithms to choose from in R.
- `optim()` is a great starting place for learning about optimization!

`optim()` manages all book keeping for us.

- `optim()` uses finite differences to approximate the derivatives, so you don't even need to perform the differentiation!
- It even calculates the Hessian matrix for us!
- We just have to supply a function to evaluate.

Finite differences approximate derivatives by applying a small perturbation to a variable

- Consider a forward difference to μ .
- Calculate the log-posterior at a reference value, (μ, σ) .
- Apply a perturbation or small change, $\mu + \Delta\mu$, calculate the log-posterior at this new value.
- Approximate the first derivative as:

$$\frac{\partial}{\partial \mu} (\log[p(\boldsymbol{\theta}|\mathbf{x})]) = \frac{\partial}{\partial \mu} (\log[p(\{\mu, \sigma\} | \mathbf{x})]) \approx \frac{\log[p(\{\mu + \Delta\mu, \sigma\} | \mathbf{x})] - \log[p(\{\mu, \sigma\} | \mathbf{x})]}{\Delta\mu}$$

Similar sequence of steps when estimating the derivative with respect to σ

- Consider a forward difference to σ .
- Calculate the log-posterior at a reference value, (μ, σ) .
- Apply a perturbation or small change, $\sigma + \Delta\sigma$, calculate the log-posterior at this new value.
- Approximate the first derivative as:

$$\frac{\partial}{\partial \sigma} (\log[p(\boldsymbol{\theta}|\mathbf{x})]) = \frac{\partial}{\partial \sigma} (\log[p(\{\mu, \sigma\} | \mathbf{x})]) \approx \frac{\log[p(\{\mu, \sigma + \Delta\sigma\} | \mathbf{x})] - \log[p(\{\mu, \sigma\} | \mathbf{x})]}{\Delta\sigma}$$

The Hessian matrix requires second order derivatives to be approximated!

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2}{\partial \mu^2} (\log[p(\{\mu, \sigma\} | \mathbf{x})]) & \frac{\partial^2}{\partial \mu \partial \sigma} (\log[p(\{\mu, \sigma\} | \mathbf{x})]) \\ \frac{\partial^2}{\partial \sigma \partial \mu} (\log[p(\{\mu, \sigma\} | \mathbf{x})]) & \frac{\partial^2}{\partial (\sigma)^2} (\log[p(\{\mu, \sigma\} | \mathbf{x})]) \end{bmatrix}$$

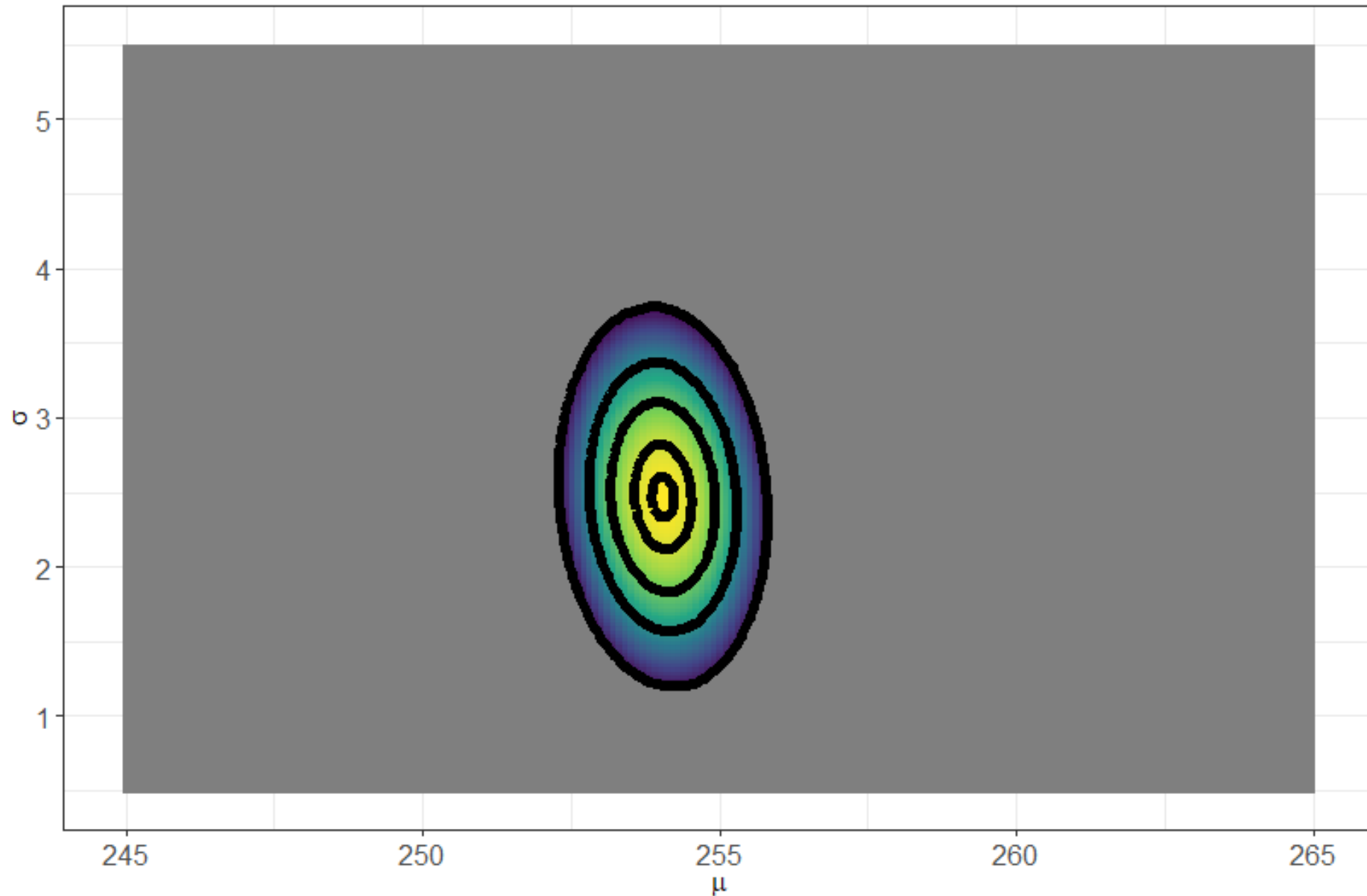
The Hessian matrix is computationally expensive to calculate!

Let's see the result of the Laplace Approximation

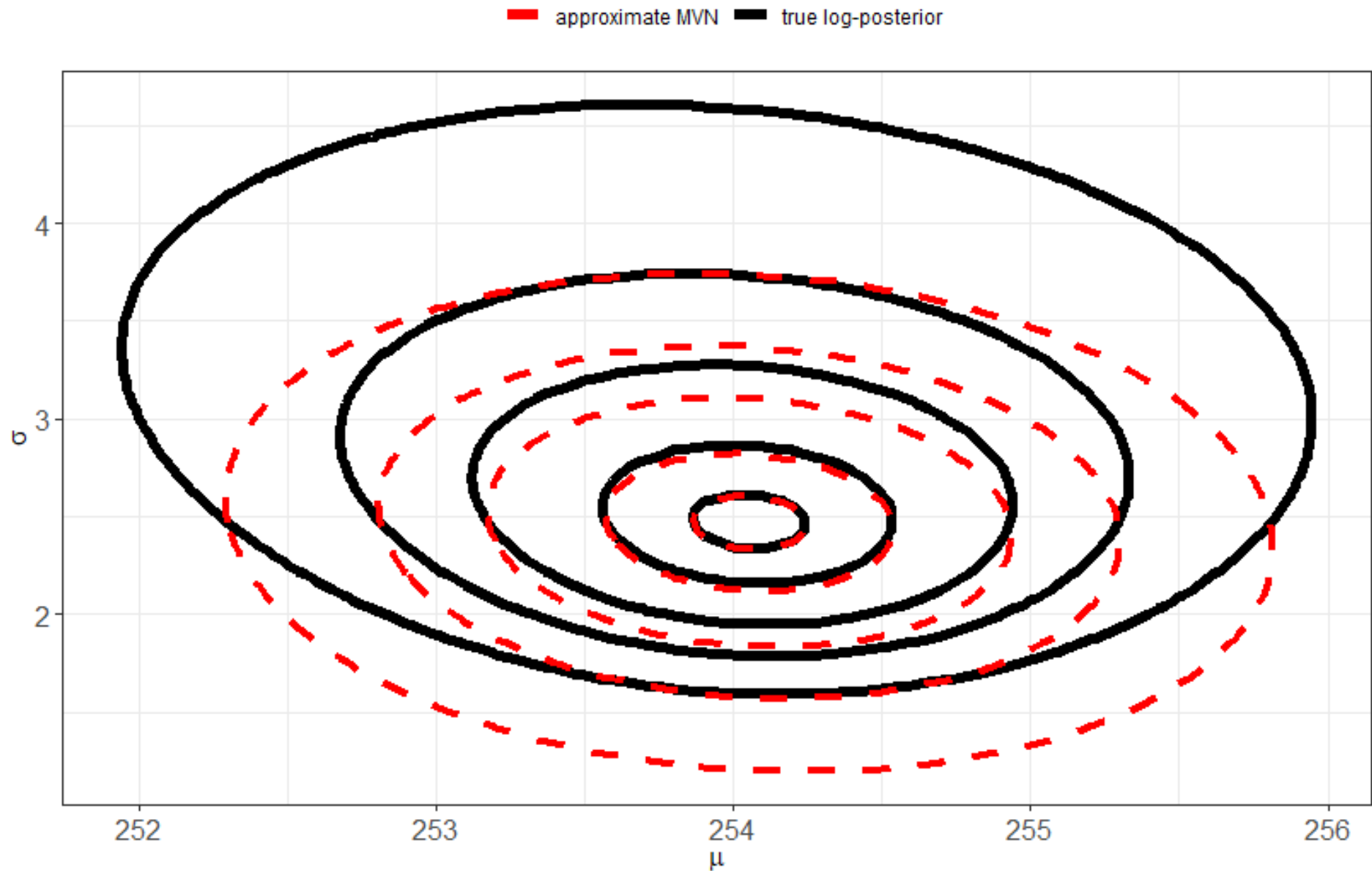
- We will go over the details of programming the Laplace Approximation in $\mathbb{R}$ later.
- After performing the Laplace Approximation we can visualize the approximate joint posterior with the MVN density.

$$p(\boldsymbol{\theta}|\mathbf{x}) \approx \mathcal{N}\left(\hat{\boldsymbol{\theta}}, \left[-\mathbf{H}\big|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}\right]^{-1}\right)$$

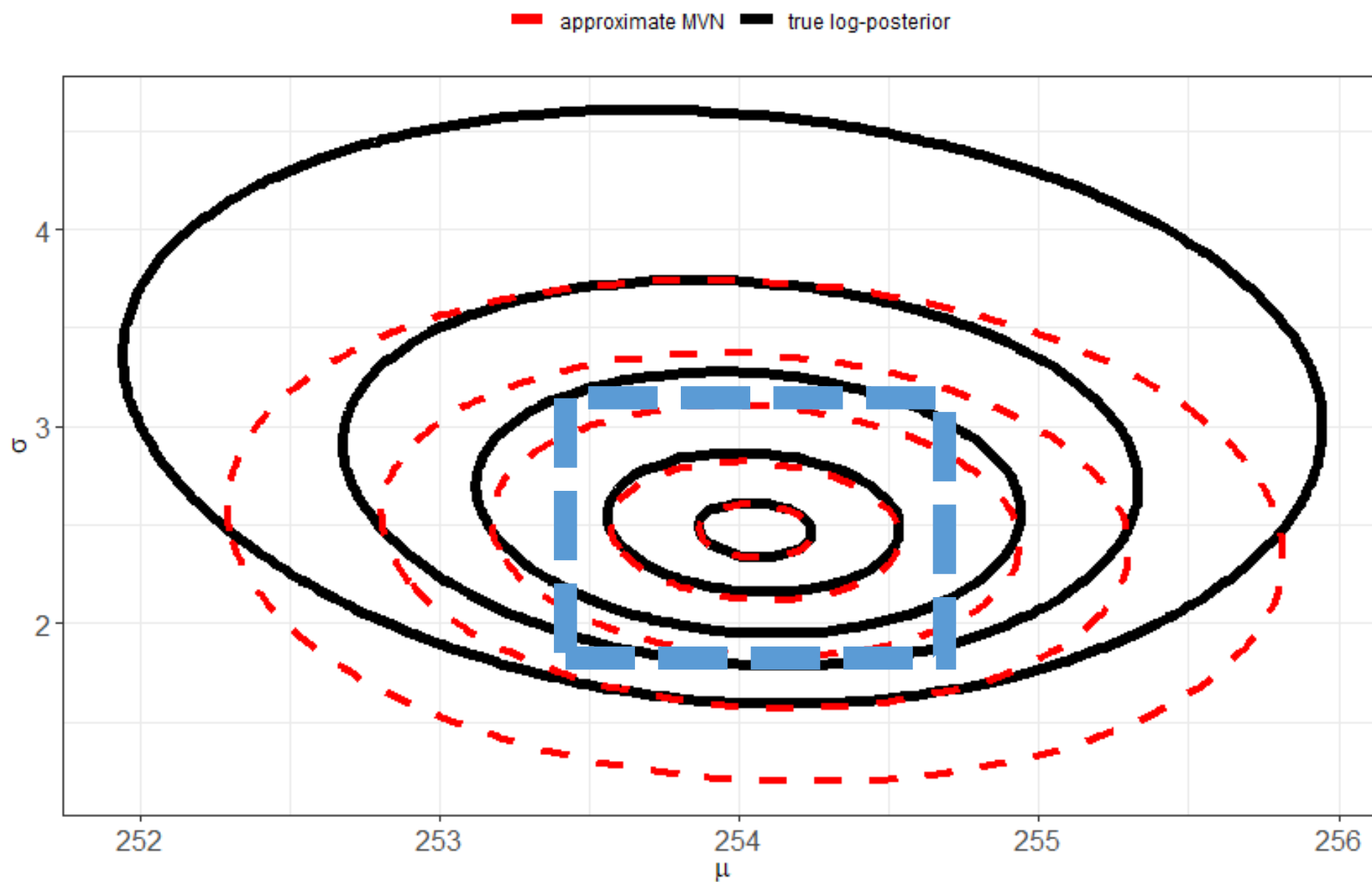
Approximate log-posterior based on 35 observations



Zoom in and compare the approximate and true log-posteriors

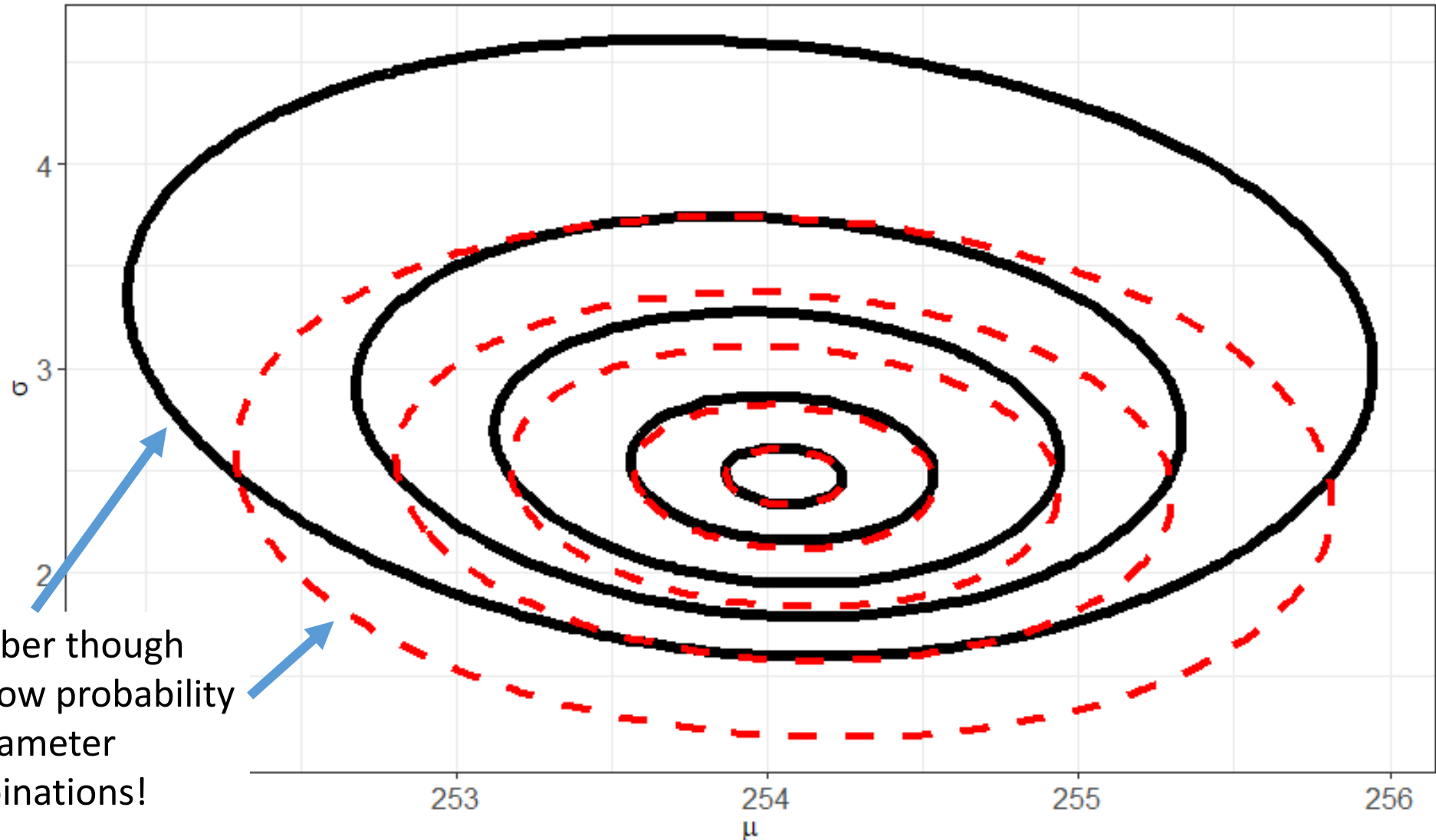


The Laplace Approximation captures the behavior around the MAP very well!



However, the true log-posterior is not symmetric around the MAP.
The approximation has trouble with the outer most quantiles, because the MVN is symmetric!

— approximate MVN — true log-posterior

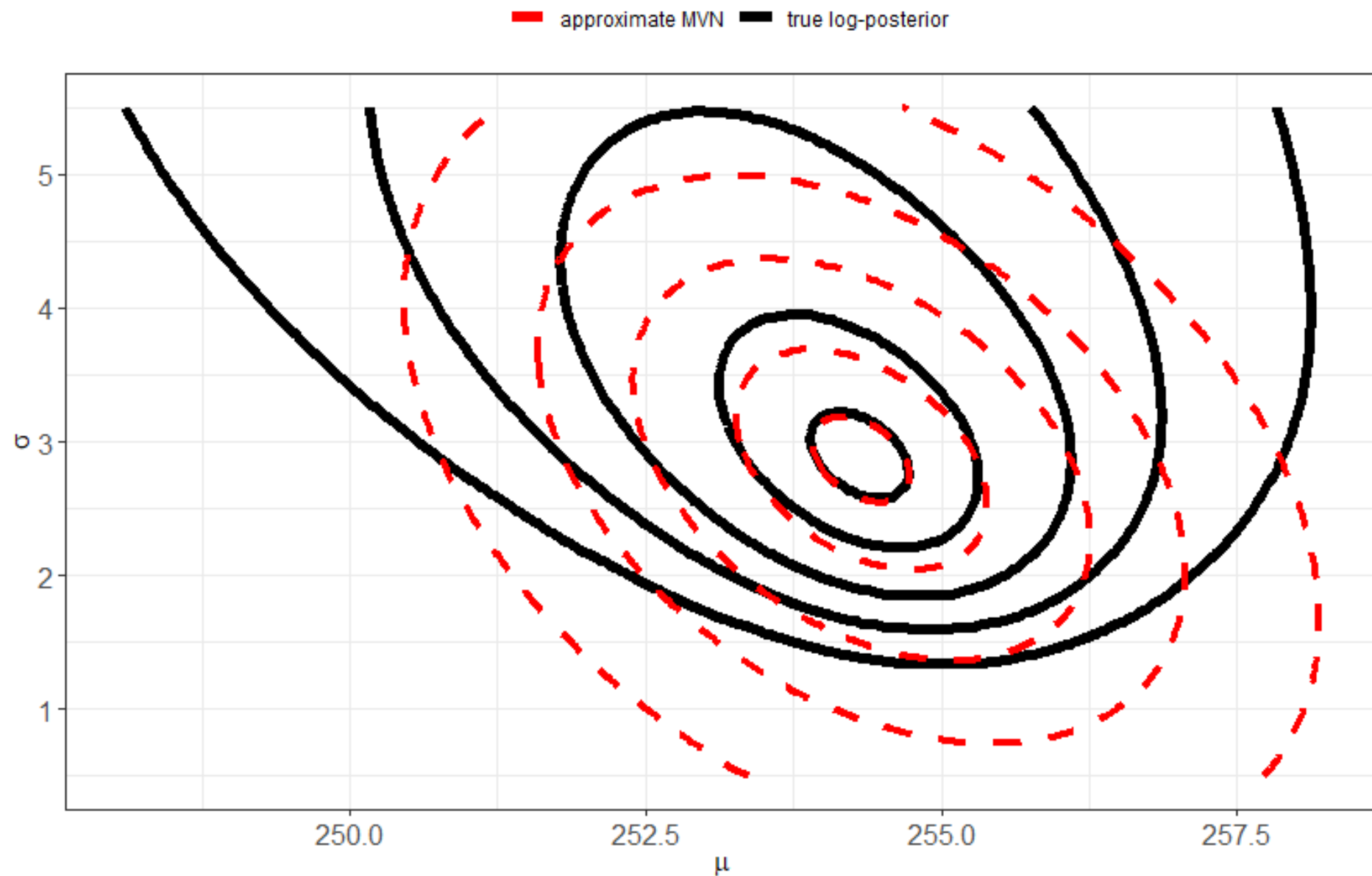


Remember though
these are low probability
parameter
combinations!

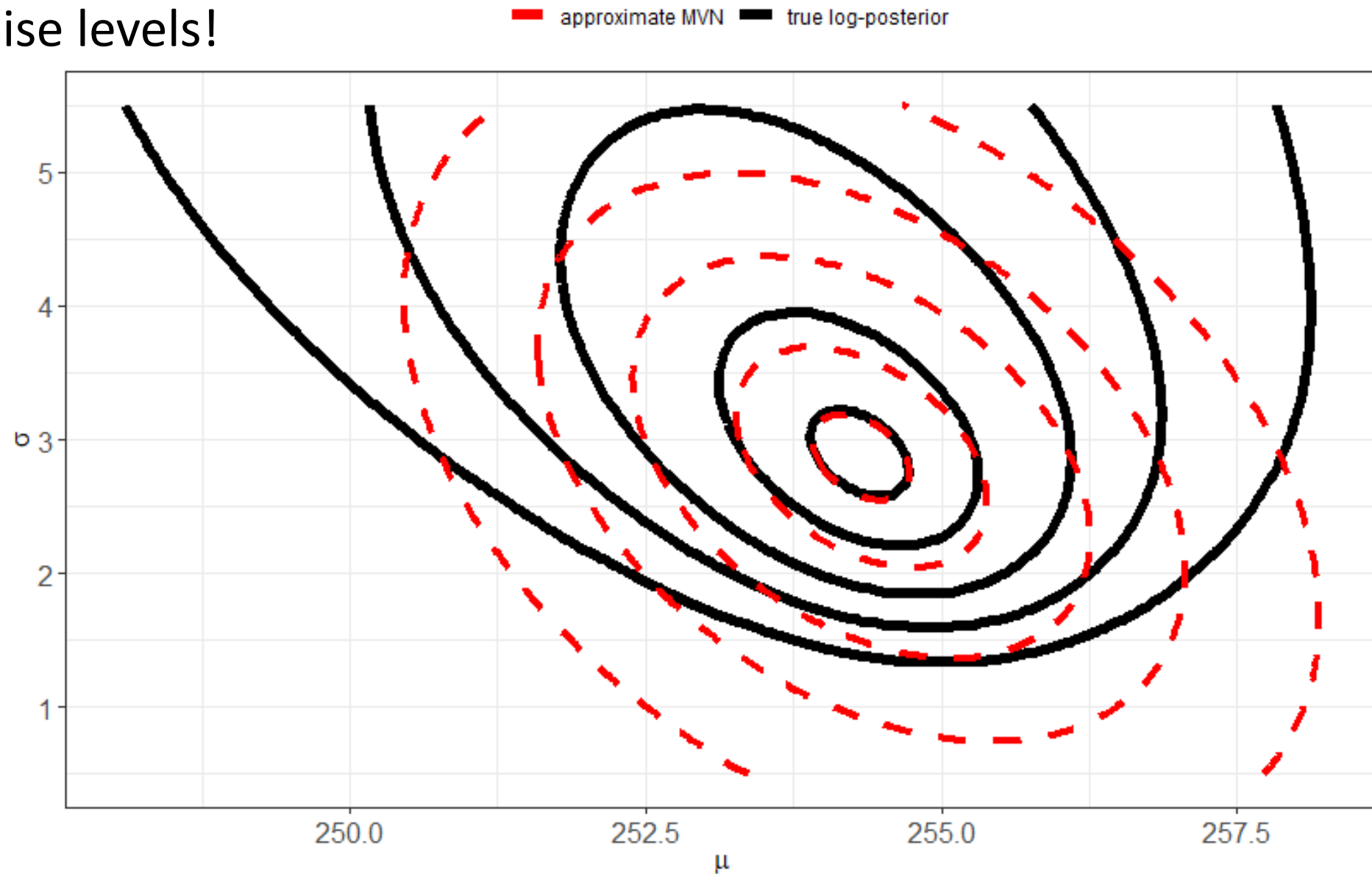
How does the Laplace approximation perform as we REDUCE the number of observations?

- Let's compare the Laplace approximate posterior to the actual posterior based on:
 - 10 observations
 - 5 observations

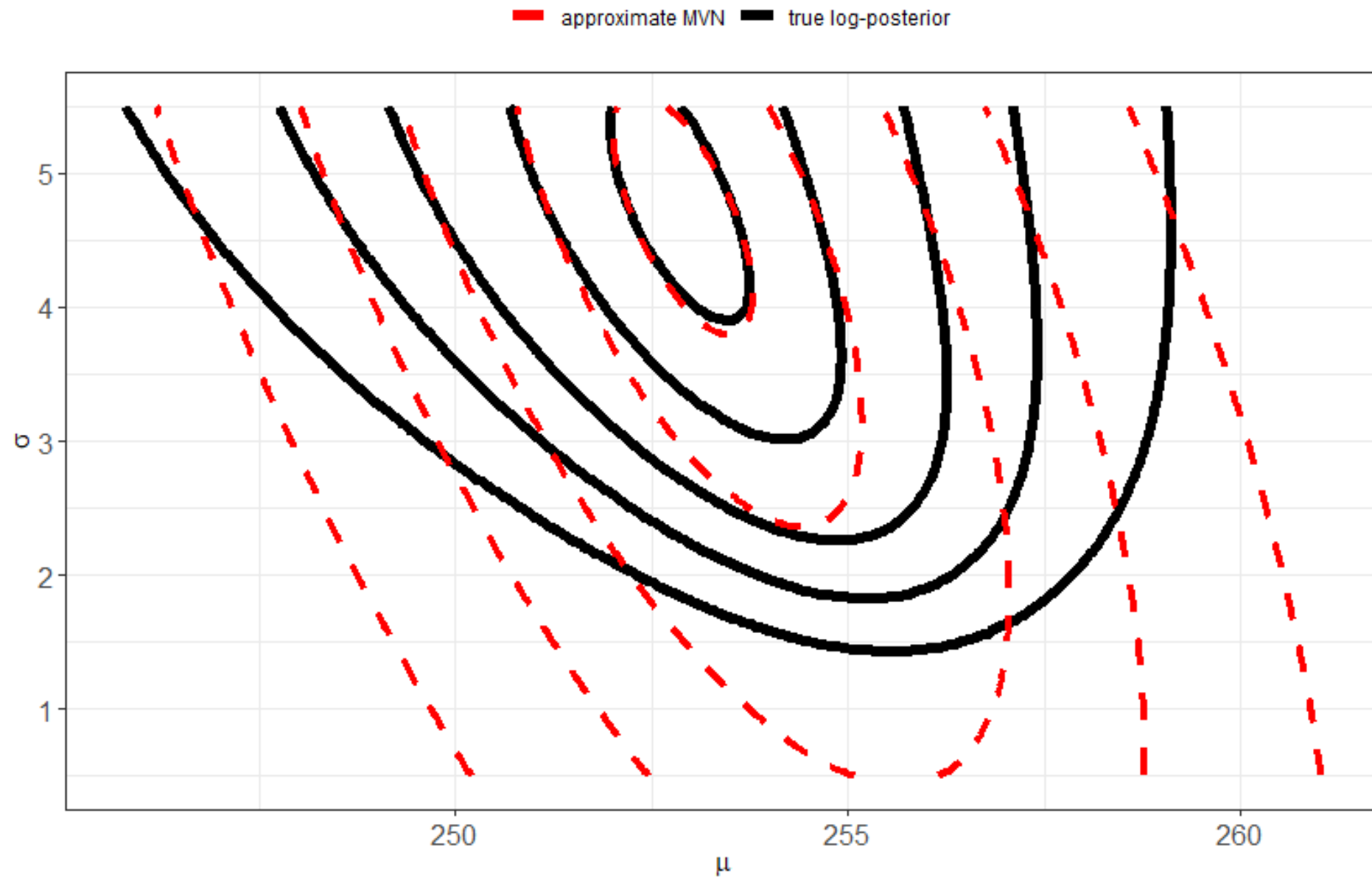
With 10 observations...



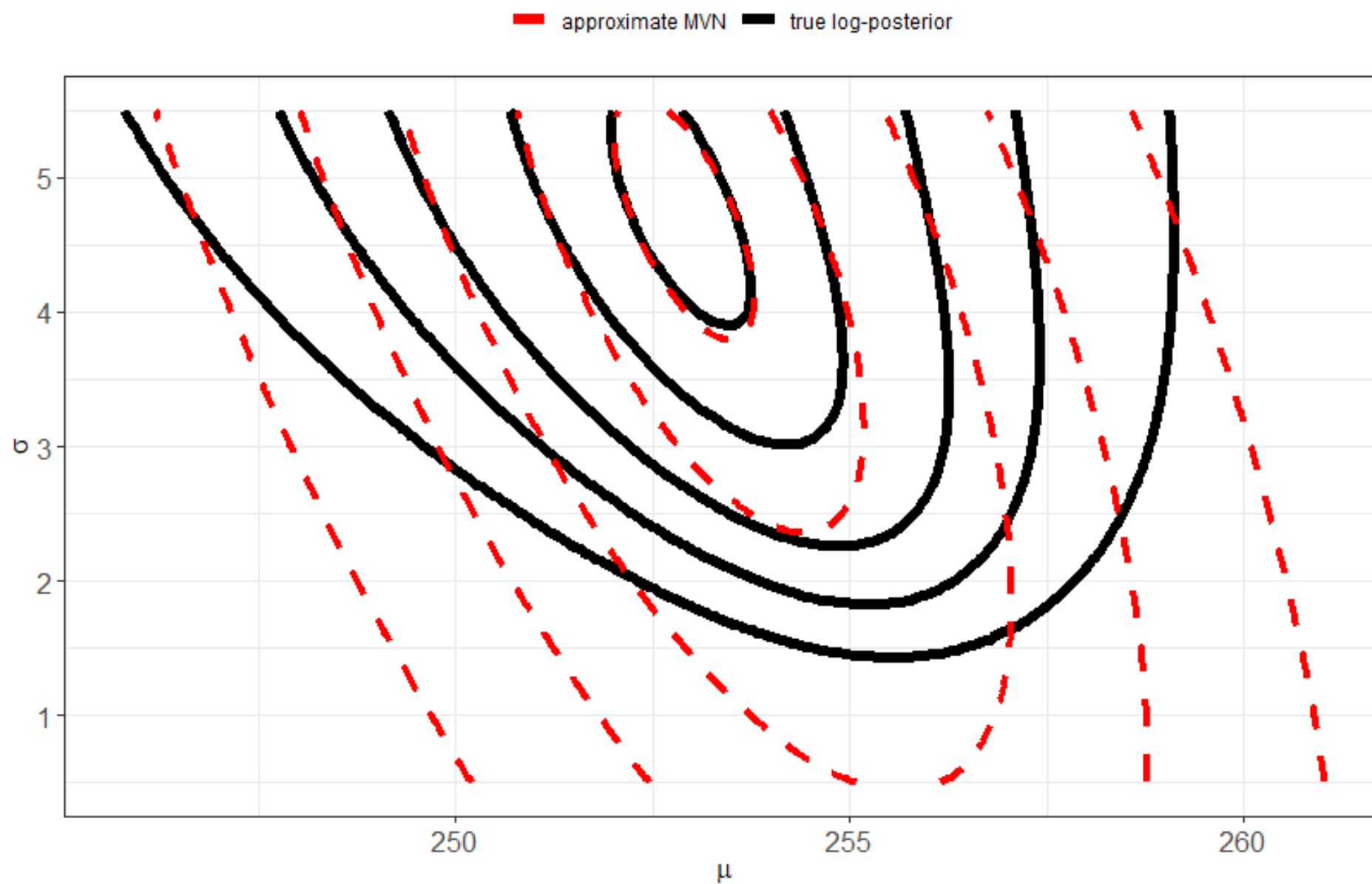
The fact that the approximation is symmetric is really starting to hurt us here...the Laplace Approximation result does not completely rule out the low noise levels!



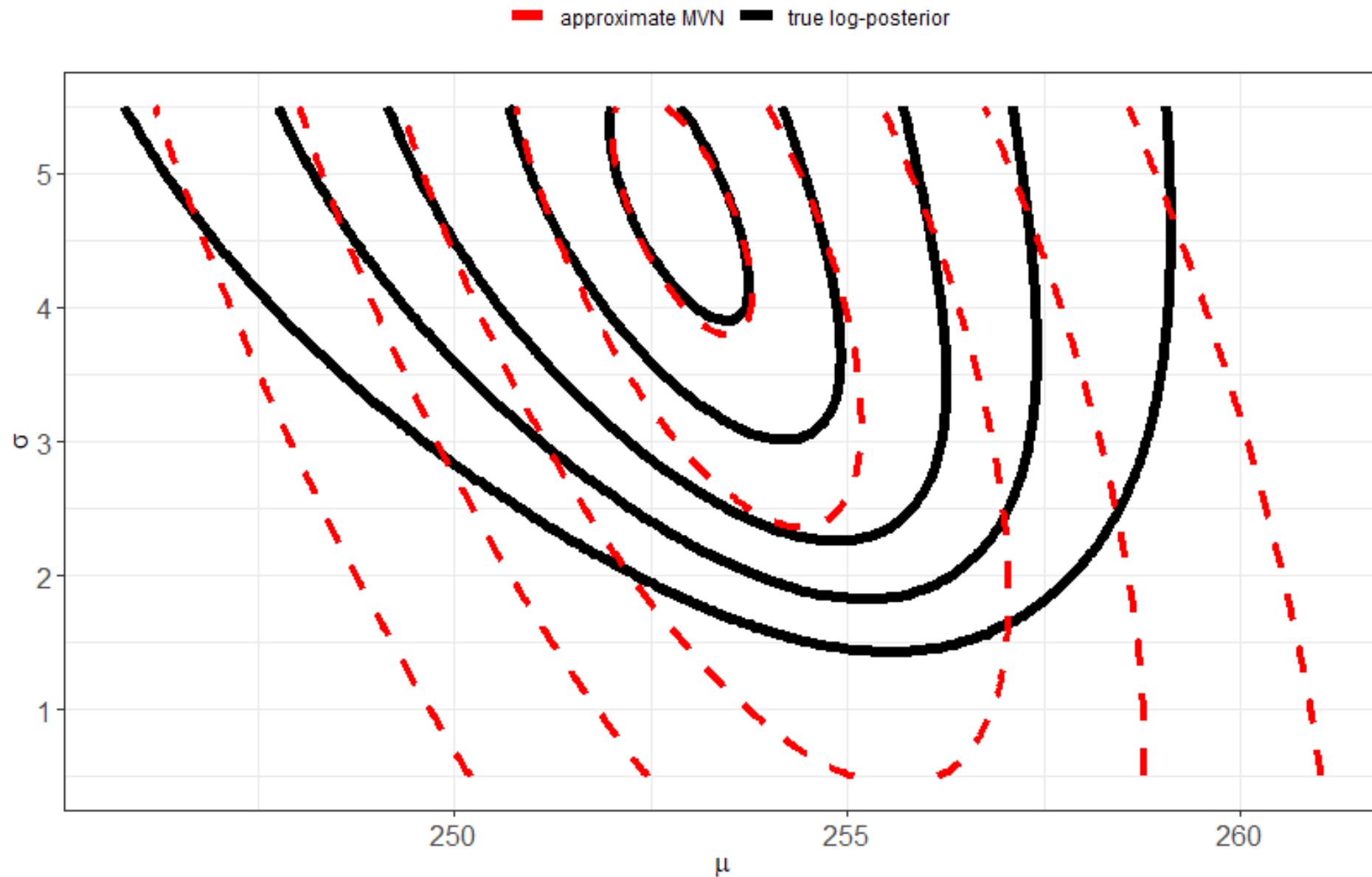
With just 5 observations...the approximation seems quite poor



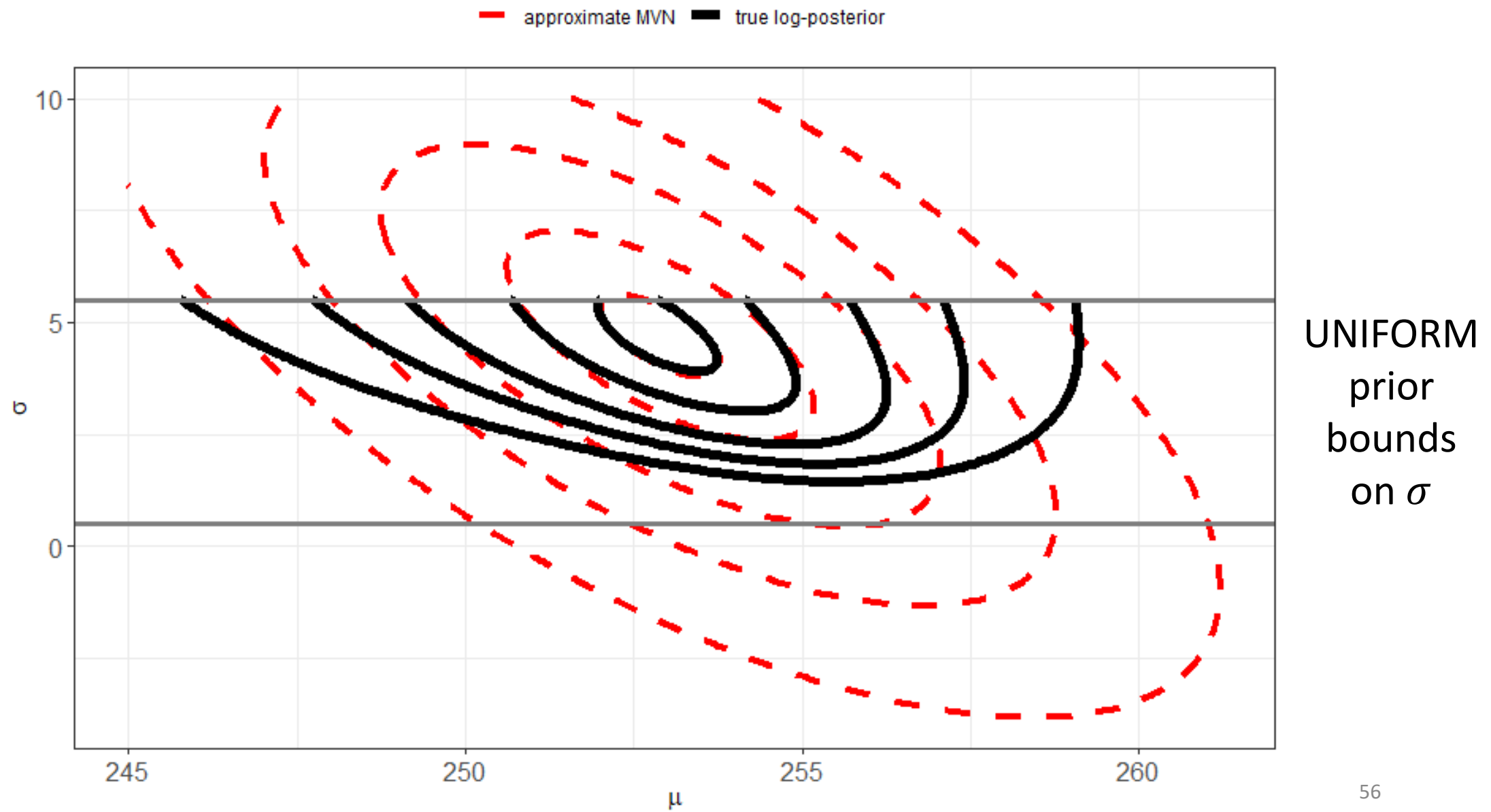
Contours with respect to μ seem consistent with the truth...



However the contours with respect to σ are only representative close to the MAP...



That's not even the worse aspect...in truth σ is allowed to be negative!!!



What's going on?!?!

- Each variable in a Multivariate Normal distribution has a Gaussian distribution.
- **Gaussian variables are unbounded: $-\infty \rightarrow +\infty$ are allowed!**
- Even though the PRIOR on σ is UNIFORM...the approximate Gaussian posterior does not care about such bounds.
- The natural lower 0 bound on σ is also ignored!

How can we satisfy the imposed UNIFORM constraint?

- Apply a transformation!
- Use the change-of-variables procedure to transform from the bounded σ to a new unbounded variable ϕ .

$$\sigma = g^{-1}(\phi), \phi = g(\sigma)$$

- Apply the Laplace approximation to the joint posterior distribution on μ, ϕ since both are unbounded.

With the change-of-variables procedure we can still use our original prior on σ

The joint-posterior on μ, ϕ can therefore be written based on the joint posterior μ, σ :

$$p(\mu, \phi | \mathbf{x}) = \prod_{n=1}^N \{ \underbrace{p(x_n | \mu, \underbrace{g^{-1}(\phi)}_{\sigma})}_{\mathcal{L}} \} \underbrace{p(\mu | \mu_0, \tau_0)}_{\mathcal{P}} \underbrace{p(\underbrace{g^{-1}(\phi)}_{\sigma} | l, u)}_{\mathcal{G}} \cdot \left| \frac{d}{d\phi} (g^{-1}(\phi)) \right|$$

With the change-of-variables procedure we can still use our original prior on σ

The joint-posterior on μ, ϕ can therefore be written based on the joint posterior μ, σ :

$$p(\mu, \phi | \mathbf{x}) = \underbrace{\prod_{n=1}^N \{p(x_n | \mu, g^{-1}(\phi))\} p(\mu | \mu_0, \tau_0) p(g^{-1}(\phi) | l, u)}_{\text{"Back-transform" } \phi \text{ to } \sigma \text{ and substitute into the joint-posterior just as before when we were working with } \sigma \text{ directly.}} \cdot \left| \frac{d}{d\phi} (g^{-1}(\phi)) \right|$$

“Back-transform” ϕ to σ and substitute into the joint-posterior just as before when we were working with σ directly.

With the change-of-variables procedure we can still use our original prior on σ

The joint-posterior on μ, ϕ can therefore be written based on the joint posterior μ, σ :

$$p(\mu, \phi | \mathbf{x}) = \prod_{n=1}^N \{p(x_n | \mu, g^{-1}(\phi))\} p(\mu | \mu_0, \tau_0) p(g^{-1}(\phi) | l, u) \cdot \left| \frac{d}{d\phi} (g^{-1}(\phi)) \right|$$

Derivative of the
transformation
with respect to ϕ

Use a logit-transformation to satisfy the lower and upper bounds on σ

- The logit, or log-odds transformation maps a lower and upper bounded variable to an unbounded variable.

$$\phi = \text{logit}(\psi) = \log \left[\frac{\psi}{1 - \psi} \right]$$

$$\psi = \text{logit}^{-1}(\phi) = \text{logistic}(\phi) = \frac{1}{1 + \exp(-\phi)} = \frac{\exp(\phi)}{1 + \exp(\phi)}$$

- **We will discuss the logit-transformation in more detail when we talk about classification.**

Use a logit-transformation to satisfy the lower and upper bounds on σ

Transformation

$$\phi = \text{logit} \left(\frac{\sigma - l}{u - l} \right) = g(\sigma)$$

Inverse transformation

$$\sigma = l + (u - l) \cdot \text{logit}^{-1}(\phi) = g^{-1}(\phi)$$

Derivative

$$\frac{d\sigma}{d\phi} = (u - l) \cdot \text{logit}^{-1}(\phi) \cdot (1 - \text{logit}^{-1}(\phi))$$

Even though `optim()` takes care of the gradient evaluations for us...we MUST supply the change of variables derivative because we are changing the function!

Transformation

$$\phi = \text{logit} \left(\frac{\sigma - l}{u - l} \right) = g(\sigma)$$

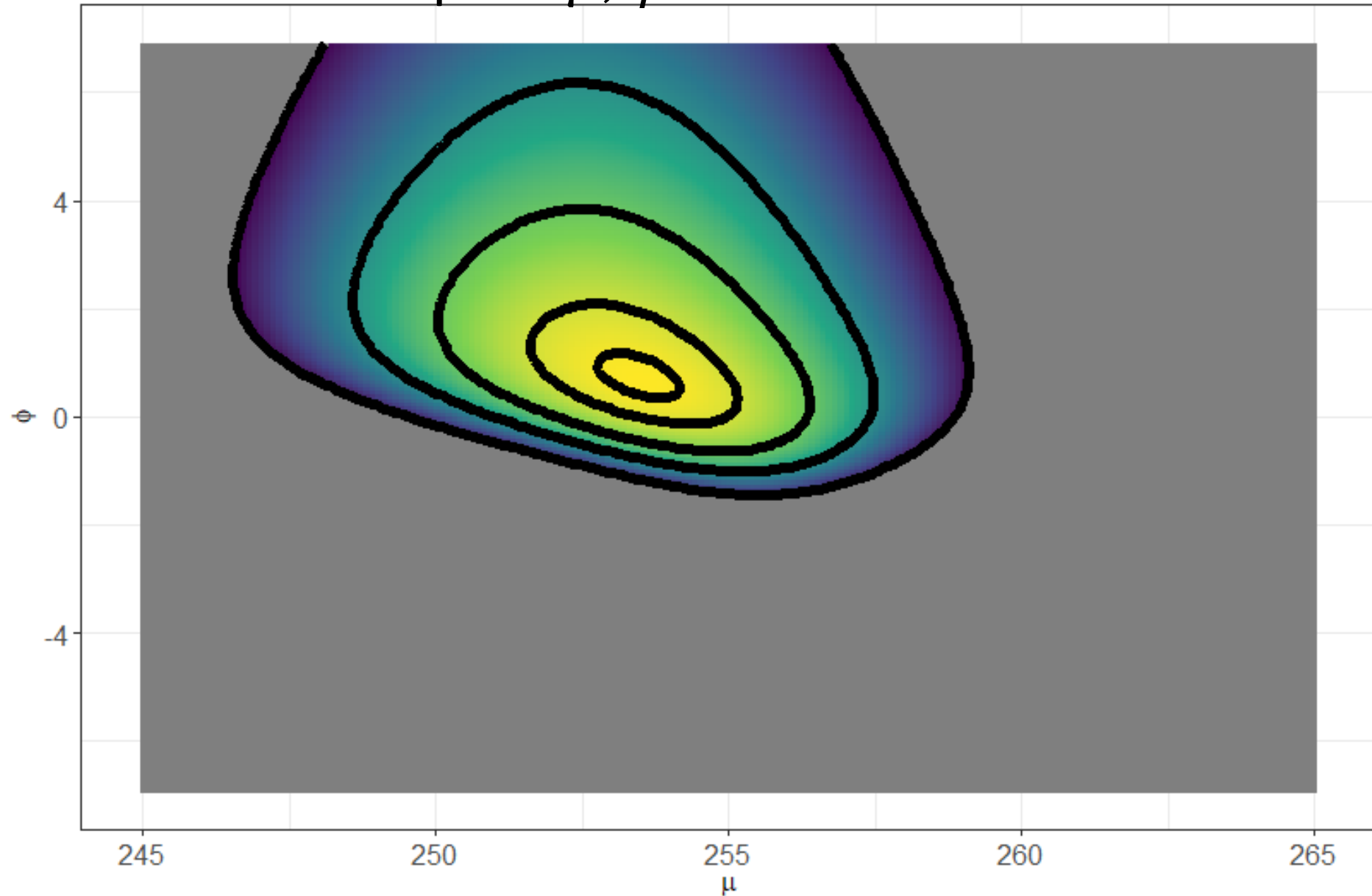
Inverse transformation

$$\sigma = l + (u - l) \cdot \text{logit}^{-1}(\phi) = g^{-1}(\phi)$$

Derivative

$$\frac{d\sigma}{d\phi} = (u - l) \cdot \text{logit}^{-1}(\phi) \cdot (1 - \text{logit}^{-1}(\phi))$$

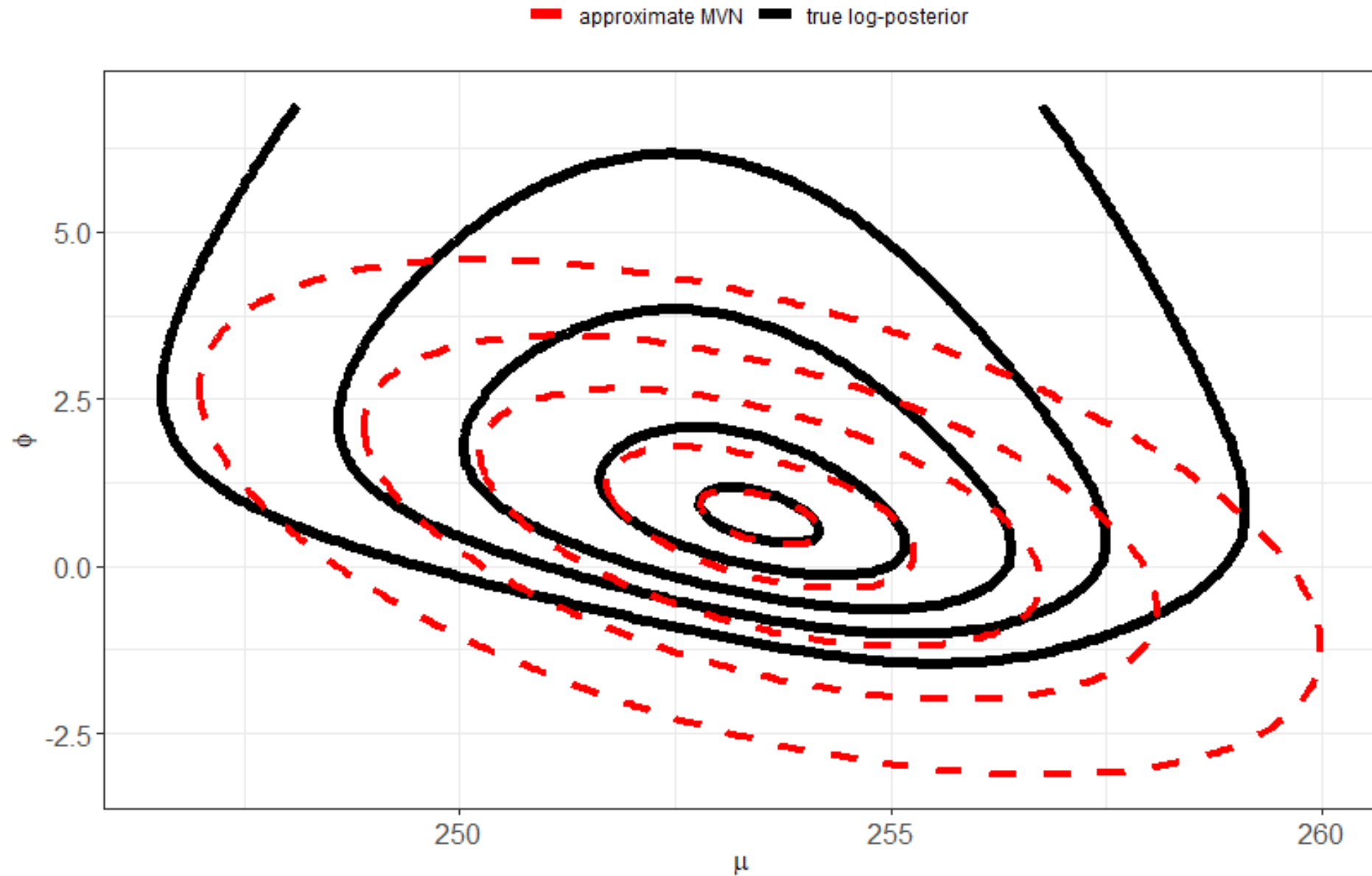
Visualize the log-posterior surface in the transformed unbounded space μ, ϕ with $N = 5$



Perform the Laplace Approximation on the transformed unbounded μ, ϕ posterior

- Consider the case with $N = 5$.
- We want to approximate the log-density surface on the previous slide.

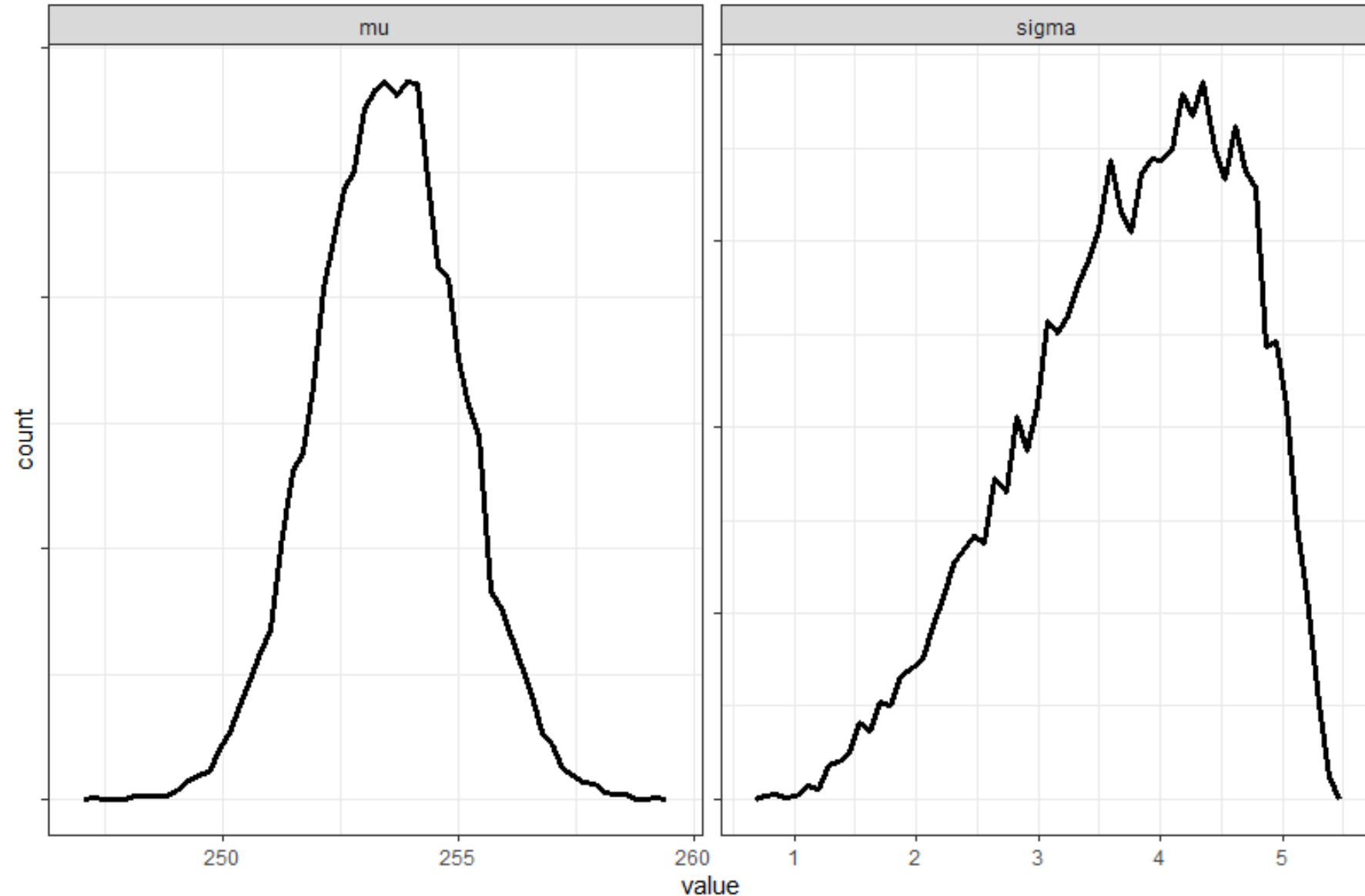
Ok...but what does this mean in terms of σ ?



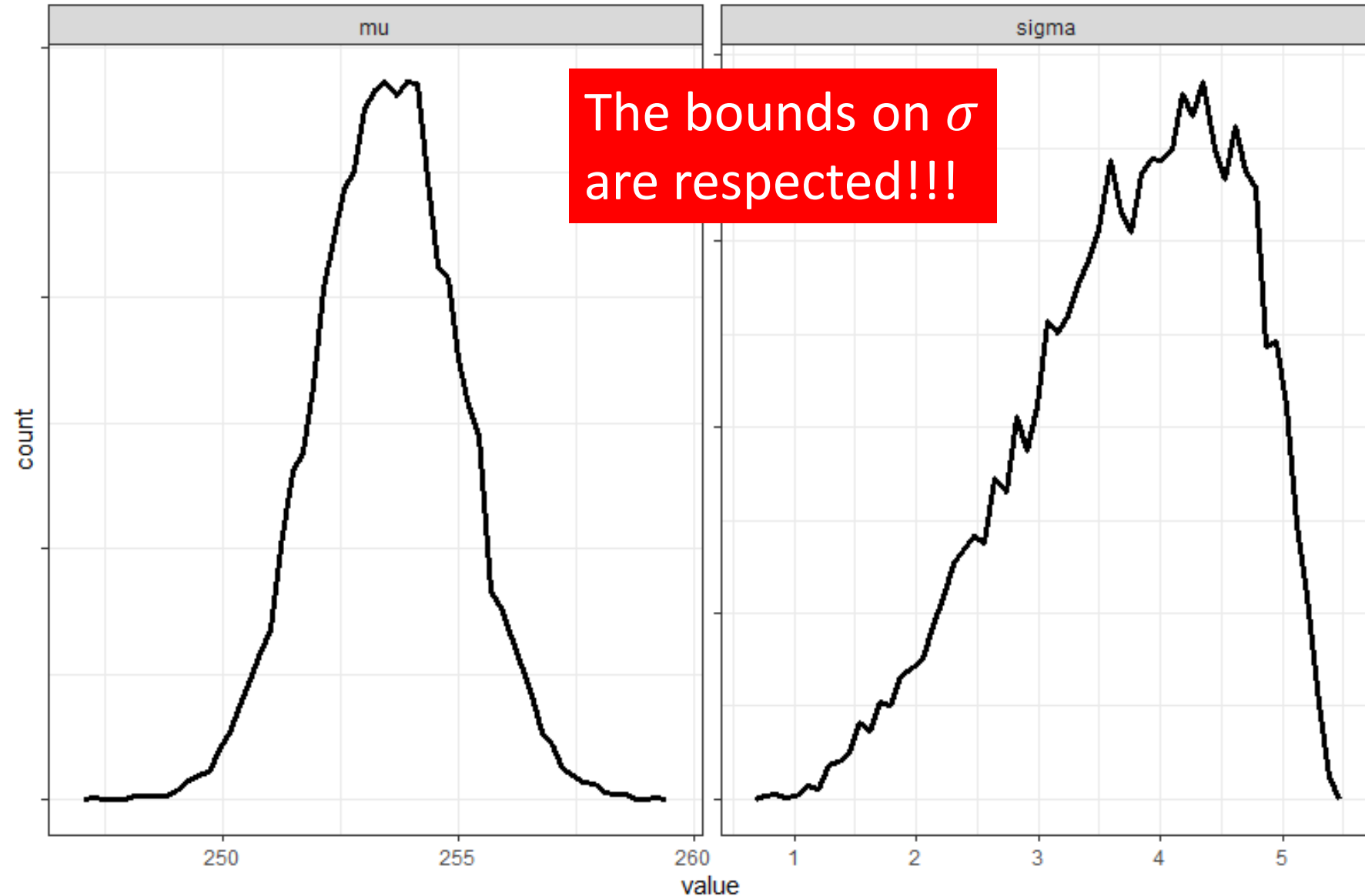
Back-transforming from ϕ to σ is difficult to do...

- However, it's straight forward to do with random sampling!
- Generate random samples from the MVN approximate posterior $p(\mu, \phi \mid \mathbf{x})$.
- Back-transform the ϕ samples to σ through the inverse transformation function:
$$\sigma = l + (u - l) \cdot \text{logit}^{-1}(\phi)$$

Posterior histograms of the original parameters from the Transformed Laplace Approximation and $N = 5$



Posterior histograms of the original parameters from the Transformed Laplace Approximation and $N = 5$



Highlights of the Laplace Approximation

- Approximates a joint distribution by a MVN.
- **Applicable when the joint distribution is unimodal!**
- Works best with moderate to large sample sizes.
- Requires unbounded parameters.
- Can generalize to bounded or skewed distributions via transformations!

Why do I like the Laplace Approximation?

- It is a microcosm for machine learning!
- The true answer might be too difficult to reach, so we need an approximation.
- We must make tradeoffs, and we need to understand the limitations of our approach.
- However, **an approximate answer to the right question is better than an exact answer to a wrong one!**

In lecture, we looked at the expression for the un-normalized log-posterior:

$$\log[p(\mu, \sigma \mid \mathbf{x})] \propto \sum_{n=1}^N (\log[\text{normal}(x_n \mid \mu, \sigma)]) + \log[\text{normal}(\mu \mid \mu_0, \tau_0)] + \log[\text{uniform}(\sigma \mid l, u)]$$

We need to turn those expressions into code!

In R, define the function `my_logpost()`

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                       mean = lik_mu,
58                       sd = lik_sigma,
59                       log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                     mean = my_info$mu_0,
64                     sd = my_info$tau_0,
65                     log = TRUE) +
66     dunif(x = lik_sigma,
67          min = my_info$sigma_lwr,
68          max = my_info$sigma_upr,
69          log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
```

In R, define the function `my_logpost()`

The parameters, μ and σ , are defined as the first and second elements of the `theta` vector.

The second argument, `my_info`, is a list which stores all information required to evaluate the log-posterior.

- Contains the vector `xobs` which stores the observations.
- Contains of the hyperparameters to the prior on μ , `mu_0` and `tau_0`.
- Contains the hyperparameters to the prior on σ , `sigma_lwr` and `sigma_upr`.

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                        mean = lik_mu,
58                        sd = lik_sigma,
59                        log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                     mean = my_info$mu_0,
64                     sd = my_info$tau_0,
65                     log = TRUE) +
66     dunif(x = lik_sigma,
67          min = my_info$sigma_lwr,
68          max = my_info$sigma_upr,
69          log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
74
```

In R, define the function `my_logpost()`

$$\sum_{n=1}^N \{\log[\text{normal}(x_n | \mu, \sigma)]\}$$

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                        mean = lik_mu,
58                        sd = lik_sigma,
59                        log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                      mean = my_info$mu_0,
64                      sd = my_info$tau_0,
65                      log = TRUE) +
66     dunif(x = lik_sigma,
67           min = my_info$sigma_lwr,
68           max = my_info$sigma_upr,
69           log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
74
```

In R, define the function `my_logpost()`

$$\sum_{n=1}^N \{\log[\text{normal}(x_n | \mu, \sigma)]\}$$

$$\log[\text{normal}(\mu | \mu_0, \tau_0)]$$

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                        mean = lik_mu,
58                        sd = lik_sigma,
59                        log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                      mean = my_info$mu_0,
64                      sd = my_info$tau_0,
65                      log = TRUE) +
66     dunif(x = lik_sigma,
67           min = my_info$sigma_lwr,
68           max = my_info$sigma_upr,
69           log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
74
```

In R, define the function `my_logpost()`

$$\sum_{n=1}^N \{\log[\text{normal}(x_n | \mu, \sigma)]\}$$

$$\log[\text{normal}(\mu | \mu_0, \tau_0)]$$

$$\log[\text{uniform}(\sigma | l, u)]$$

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                        mean = lik_mu,
58                        sd = lik_sigma,
59                        log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                      mean = my_info$mu_0,
64                      sd = my_info$tau_0,
65                      log = TRUE) +
66     dunif(x = lik_sigma,
67          min = my_info$sigma_lwr,
68          max = my_info$sigma_upr,
69          log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
74
```

In R, define the function `my_logpost()`

$$\sum_{n=1}^N \{\log[\text{normal}(x_n | \mu, \sigma)]\}$$

$$\log[\text{normal}(\mu | \mu_0, \tau_0)]$$

$$\log[\text{uniform}(\sigma | l, u)]$$

SUM the three pieces together
to produce the log-posterior!!!

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                        mean = lik_mu,
58                        sd = lik_sigma,
59                        log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                      mean = my_info$mu_0,
64                      sd = my_info$tau_0,
65                      log = TRUE) +
66     dunif(x = lik_sigma,
67          min = my_info$sigma_lwr,
68          max = my_info$sigma_upr,
69          log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
74
```

Define the hyperparameters and subset the observations from the vector $\mathbf{x}$

```
553 ### define the information to use for 35 observations
554 info_N35 <- list(
555   xobs = x[1:35],
556   mu_0 = 250,
557   tau_0 = 2,
558   sigma_lwr = 0.5,
559   sigma_upr = 5.5
560 )
561
```

For this specific example, I used the first 35 elements in the vector $\mathbf{x}$
Pay attention to what you need to use for the homework!

`optim()` manages the bookkeeping of the optimization algorithm for us

- Requires that the function we wish to optimize has its **FIRST** argument be a numeric vector.
- `optim()` takes care of modifying the elements of that numeric vector in order to optimize the function of interest.

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                       mean = lik_mu,
58                       sd = lik_sigma,
59                       log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                     mean = my_info$mu_0,
64                     sd = my_info$tau_0,
65                     log = TRUE) +
66     dunif(x = lik_sigma,
67          min = my_info$sigma_lwr,
68          max = my_info$sigma_upr,
69          log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
74
```

optim() manages the bookkeeping of the optimization algorithm for us

That is why `theta` is the first argument to `my_logpost()` !!!

- Requires that the function we wish to optimize has its **FIRST** argument be a numeric vector.
- `optim()` takes care of modifying the elements of that numeric vector in order to optimize the function of interest.

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                        mean = lik_mu,
58                        sd = lik_sigma,
59                        log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                     mean = my_info$mu_0,
64                     sd = my_info$tau_0,
65                     log = TRUE) +
66     dunif(x = lik_sigma,
67          min = my_info$sigma_lwr,
68          max = my_info$sigma_upr,
69          log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
74
```

Basic syntax of an `optim()` call.

```
562 ### set the initial guess  
563 init_param <- c(250, 5) We specify an initial guess.  
564  
565 ### find the posterior mode (MAP) using the optimization scheme  
566 map_result <- optim(init_param, ← That initial guess is the first  
567 my_logpost, argument to optim().  
568 gr = NULL,  
569 info_N35,  
570 method = "BFGS",  
571 hessian = TRUE,  
572 control = list(fnscale = -1, maxit = 1001))  
573
```

Basic syntax of an `optim()` call.

```
562 ### set the initial guess
563 init_param <- c(250, 5)
564
565 ### find the posterior mode (MAP) using the optimization scheme
566 map_result <- optim(init_param,
567                     my_logpost,
568                     gr = NULL,
569                     info_N35,
570                     method = "BFGS",
571                     hessian = TRUE,
572                     control = list(fnscale = -1, maxit = 1001))
573
```

Second argument is the function we wish to optimize.

Basic syntax of an `optim()` call.

```
562 ### set the initial guess
563 init_param <- c(250, 5)
564
565 ### find the posterior mode (MAP)
566 map_result <- optim(init_param,
567                     my_logpost,
568                     gr = NULL,
569                     info_N35,
570                     method = "BFGS",
571                     hessian = TRUE,
572                     control = list(),
573
```

Third argument is a **function** which returns the gradient vector. If we require the gradient to be evaluated **numerically**, set to `NULL`

Basic syntax of an `optim()` call.

```
562 ### set the initial guess
563 init_param <- c(250, 5)
564
565 ### find the posterior mode (MAP)
566 map_result <- optim(init_param,
567                     my_logpost,
568                     gr = NULL,
569                     info_N35,
570                     method = "BFGS",
571                     hessian = TRUE,
572                     control = list(),
573
```

After the `gr` argument, we can pass in **ALL additional inputs** required to evaluate the function we wish to optimize.

Basic syntax of an `optim()` call.

```
562 ### set the initial guess
563 init_param <- c(250, 5)
564
565 ### find the posterior mode (MAP)
566 map_result <- optim(init_param,
567                     my_logpost,
568                     gr = NULL,
569                     info_N35,
570                     method = "BFGS",
571                     hessian = TRUE,
572                     control = list(),
573
```

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                        mean = lik_mu,
58                        sd = lik_sigma,
59                        log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                      mean = my_info$mu_0,
64                      sd = my_info$tau_0,
65                      log = TRUE) +
66     dunif(x = lik_sigma,
67           min = my_info$sigma_lwr,
68           max = my_info$sigma_upr,
69           log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
74
```

Basic syntax of an `optim()` call.

After setting the `gr` argument, all remaining arguments **MUST** be named.

```
562 ### set the initial guess  
563 init_param <- c(250, 5)  
564  
565 ### find the posterior mode (MAP) using the optimization scheme  
566 map_result <- optim(init_param,  
567                       my_logpost,  
568                       gr = NULL,  
569                       info_N35,  
570                       method = "BFGS",  
571                       hessian = TRUE,  
572                       control = list(fnscale = -1, maxit = 1001))  
573
```


Basic syntax of an `optim()` call.

After setting the `gr` argument, all remaining arguments **MUST** be named.

```
562 ### set the initial guess  
563 init_param <- c(250, 5)  
564 ### fit the posterior mode (MAP) using the optimization scheme  
565 optim(init_param,  
566       my_logpost,  
567       gr = NULL,  
568       info_N35,  
569       method = "BFGS",  
570       hessian = TRUE,  
571       control = list(fnscale = -1, maxit = 1001))
```

Set which optimization method to use. By default `optim()` uses Nelder-Mead. I like to use the Quasi-Newton BFGS algorithm. Try out different methods, see which one you prefer.

We will discuss the BFGS in a little more detail later in the semester

Basic syntax of an `optim()` call.

After setting the `gr` argument, all remaining arguments **MUST** be named.

```
562 ### set the initial guess
563 init_param <- c(250, 5)
564
565 ### find the posterior mode (MAP) using the optimization scheme
566 map_result <- optim(init_param,
567                     my_logpost,
                     gr = NULL,
                     info_N35,
                     method = "BFGS",
                     hessian = TRUE,
                     control = list(fnscale = -1, maxit = 1001))
```

Tell `optim()` to compute and return the hessian matrix by setting the `hessian` flag to `TRUE`.

Basic syntax of an `optim()` call.

After setting the `gr` argument, all remaining arguments **MUST** be named.

```
562 ### set the initial guess
563 init_param <- c(250, 5)
564
565 ### find the posterior mode (MAP) using the optimization scheme
566 optim(init_param,
      my_logpost,
      gr = NULL,
      info_N35,
      method = "BFGS",
      hessian = TRUE,
      control = list(fnscale = -1, maxit = 1001))
```

`control` is a list of parameters which dictate important operating behavior of the algorithm.

See `?optim` for a complete list of all available control parameters.

Basic syntax of an `optim()` call.

After setting the `gr` argument, all remaining arguments **MUST** be named.

```
562 ### set the initial guess
563 init_param <- c(250, 5)
564
565 ### find the posterior mode (MAP) using the optimization scheme
566 optim(init_param,
        my_logpost,
        gr = NULL,
        info_N35,
        method = "BFGS",
        hessian = TRUE,
        control = list(fnscale = -1, maxit = 1001))
```

`control` is a list of parameters which dictate important operating behavior of the algorithm.

See `?optim` for a complete list of all available control parameters.

By default `optim()` seeks to **MINIMIZE** a function, so to tell it to **MAXIMIZE** set the `fnscale` control parameter to `-1`.

Basic syntax of an `optim()` call.

After setting the `gr` argument, all remaining arguments **MUST** be named.

```
562 ### set the initial guess
563 init_param <- c(250, 5)
564
565 ### find the posterior mode (MAP) using the optimization scheme
566 optim(init_param,
        my_logpost,
        gr = NULL,
        info_N35,
        method = "BFGS",
        hessian = TRUE,
        control = list(fnscale = -1, maxit = 1001))
```

`control` is a list of parameters which dictate important operating behavior of the algorithm.

See `?optim` for a complete list of all available control parameters.

`maxit` controls the maximum number of iterations, default for **derivative** based method is 100.

optim() result

```
> map_result
$par
[1] 254.053269 2.469113

$value
[1] -86.57307

$counts
function gradient
      25      10

$convergence
[1] 0

$message
NULL

$hessian
      [,1] [,2]
[1,] -5.9909797 -0.8207917
[2,] -0.8207917 -11.4819863
```

optim() result

```
> map_result
```

```
$par
```

```
[1] 254.053269 2.469113
```

Identified optimal values for the parameters

```
$value
```

```
[1] -86.57307
```

Function value at the optimal parameter settings

```
$counts
```

```
function gradient  
25 10
```

How many times was the function evaluated?

```
$convergence
```

```
[1] 0
```

Did it converge? 0 = YES! (odd choice I know...)

```
$message
```

```
NULL
```

Can ignore if it converged

```
$hessian
```

```
      [,1] [,2]  
[1,] -5.9909797 -0.8207917  
[2,] -0.8207917 -11.4819863
```

The Hessian matrix evaluated at the optimal parameter settings

To complete the Laplace approximation...

- Define a wrapper which executes the `optim()` call and then calculates the remaining pieces of the Laplace approximation.
- The following code is adapted from the `laplace()` function from the `LearnBayes` package by Jim Albert.
- Great book! [Bayesian Computation with R](#)

Can you decipher what's happening in this function?

```
528 ### define a function for performing the laplace approximation
529 my_laplace <- function(start_guess, logpost_func, ...)
530 {
531   # code adapted from the `LearnBayes` function `laplace()`
532   fit <- optim(start_guess,
533               logpost_func,
534               gr = NULL,
535               ...,
536               method = "BFGS",
537               hessian = TRUE,
538               control = list(fnscale = -1, maxit = 1001))
539
540   mode <- fit$par
541   h <- -solve(fit$hessian)
542   p <- length(mode)
543   int <- p/2 * log(2 * pi) + 0.5 * log(det(h)) + logpost_func(mode, ...)
544   list(mode = mode,
545        var_matrix = h,
546        log_evidence = int,
547        converge = ifelse(fit$convergence == 0,
548                          "YES",
549                          "NO"),
550        iter_counts = fit$counts[1])
551 }
```

Can you decipher what's happening in this function?

```
528 ### define a function for performing the laplace approximation
529 my_laplace <- function(start_guess, logpost_func, ...)
530 {
531   # code adapted from the `LearnBayes` function `laplace()`
532   fit <- optim(start_guess,
533               logpost_func,
534               gr = NULL,
535               ...,
536               method = "BFGS",
537               hessian = TRUE,
538               control = list(fnscale = -1, maxit = 1001))
539
540   mode <- fit$par
541   h <- -solve(fit$hessian)
542   p <- length(mode)
543   int <- p/2 * log(2 * pi) + 0.5 * log(det(h)) + logpost_func(mode, ...)
544   list(mode = mode,
545        var_matrix = h,
546        log_evidence = int,
547        converge = ifelse(fit$convergence == 0,
548                          "YES",
549                          "NO"),
550        iter_counts = fit$counts[1])
551 }
```

Why is `fit$par`
set to be the
mode?

Can you decipher what's happening in this function?

```
528 ### define a function for performing the laplace approximation
529 my_laplace <- function(start_guess, logpost_func, ...)
530 {
531   # code adapted from the 'LearnBayes' function 'laplace()'
532   fit <- optim(start_guess,
533               logpost_func,
534               gr = NULL,
535               ...,
536               method = "BFGS",
537               hessian = TRUE,
538
539
540   mode <- fit$
541   h <- -solve(
542   p <- length(mode)
543   int <- p/2 * log(2 * pi) + 0.5 * log(det(h)) + logpost_func(mode, ...)
544   list(mode = mode,
545        var_matrix = h,
546        log_evidence = int,
547        converge = ifelse(fit$convergence == 0,
548                          "YES",
549                          "NO"),
550        iter_counts = fit$counts[1])
551 }
```

We will return to this aspect of the Laplace approximation in a future lecture!

Execute the Laplace Approximation

```
> laplace_result_N35 <- my_laplace(init_param, my_logpost, info_N35)
>
> laplace_result_N35
$mode
[1] 254.053269    2.469113

$var_matrix
      [,1]      [,2]
[1,] 0.16856853 -0.01205015
[2,] -0.01205015  0.08795435

$log_evidence
[1] -86.84579

$converge
[1] "YES"

$iter_counts
function
      25
```

Next, we need to perform the Laplace
Approximation on transformed parameters

Define a new function `my_logpost_cv()` which calculates the log-posterior on μ, ϕ

```
785 my_logpost_cv <- function(phi, my_info)
786 {
787   # the unknown mean is the first parameter
788   lik_mu <- phi[1]
789   # the unknown logit-transformed standard deviation
790   # is the second, back transform to sigma
791   lik_sigma <- my_info$sigma_lwr +
792     (my_info$sigma_upr - my_info$sigma_lwr) * boot::inv.logit(phi[2])
793
794   # log-likelihood -> sum up the independent
795   # log-likelihoods
796   log_lik <- sum(dnorm(x = my_info$xobs,
797                       mean = lik_mu,
798                       sd = lik_sigma,
799                       log = TRUE))
800
801   # the log-prior -> sum up the independent priors
802   log_prior <- dnorm(x = lik_mu,
803                     mean = my_info$mu_0,
804                     sd = my_info$tau_0,
805                     log = TRUE) +
806     dunif(x = lik_sigma,
807          min = my_info$sigma_lwr,
808          max = my_info$sigma_upr,
809          log = TRUE)
810
811   # add the log-likelihood and log-prior and account
812   # for the derivative adjustment
813   deriv_adjust <- log(my_info$sigma_upr - my_info$sigma_lwr) +
814     log(boot::inv.logit(phi[2])) +
815     log(1 - boot::inv.logit(phi[2]))
816
817   log_lik + log_prior + deriv_adjust
818 }
819
```

The log-likelihood and log-priors are evaluated just as they were in the original function!

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                       mean = lik_mu,
58                       sd = lik_sigma,
59                       log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                     mean = my_info$mu_0,
64                     sd = my_info$tau_0,
65                     log = TRUE) +
66     dunif(x = lik_sigma,
67          min = my_info$sigma_lwr,
68          max = my_info$sigma_upr,
69          log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
```

```
785 my_logpost_cv <- function(phi, my_info)
786 {
787   # the unknown mean is the first parameter
788   lik_mu <- phi[1]
789   # the unknown logit-transformed standard deviation
790   # is the second, back transform to sigma
791   lik_sigma <- my_info$sigma_lwr +
792     (my_info$sigma_upr - my_info$sigma_lwr) * boot::inv.logit(phi[2])
793
794   # log-likelihood -> sum up the independent
795   # log-likelihoods
796   log_lik <- sum(dnorm(x = my_info$xobs,
797                       mean = lik_mu,
798                       sd = lik_sigma,
799                       log = TRUE))
800
801   # the log-prior -> sum up the independent priors
802   log_prior <- dnorm(x = lik_mu,
803                     mean = my_info$mu_0,
804                     sd = my_info$tau_0,
805                     log = TRUE) +
806     dunif(x = lik_sigma,
807          min = my_info$sigma_lwr,
808          max = my_info$sigma_upr,
809          log = TRUE)
810
811   # add the log-likelihood and log-prior and account
812   # for the derivative adjustment
813   deriv_adjust <- log(my_info$sigma_upr - my_info$sigma_lwr) +
814     log(boot::inv.logit(phi[2])) +
815     log(1 - boot::inv.logit(phi[2]))
816
817   log_lik + log_prior + deriv_adjust
818 }
```

The new function “back-transforms” ϕ to σ

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                       mean = lik_mu,
58                       sd = lik_sigma,
59                       log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                     mean = my_info$mu_0,
64                     sd = my_info$tau_0,
65                     log = TRUE) +
66     dunif(x = lik_sigma,
67          min = my_info$sigma_lwr,
68          max = my_info$sigma_upr,
69          log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
```



```
785 my_logpost_cv <- function(phi, my_info)
786 {
787   # the unknown mean is the first parameter
788   lik_mu <- phi[1]
789   # the unknown logit-transformed standard deviation
790   # is the second, back transform to sigma
791   lik_sigma <- my_info$sigma_lwr +
792     (my_info$sigma_upr - my_info$sigma_lwr) * boot::inv.logit(phi[2])
793
794   # log-likelihood -> sum up the independent
795   # log-likelihoods
796   log_lik <- sum(dnorm(x = my_info$xobs,
797                       mean = lik_mu,
798                       sd = lik_sigma,
799                       log = TRUE))
800
801   # the log-prior -> sum up the independent priors
802   log_prior <- dnorm(x = lik_mu,
803                     mean = my_info$mu_0,
804                     sd = my_info$tau_0,
805                     log = TRUE) +
806     dunif(x = lik_sigma,
807          min = my_info$sigma_lwr,
808          max = my_info$sigma_upr,
809          log = TRUE)
810
811   # add the log-likelihood and log-prior and account
812   # for the derivative adjustment
813   deriv_adjust <- log(my_info$sigma_upr - my_info$sigma_lwr) +
814     log(boot::inv.logit(phi[2])) +
815     log(1 - boot::inv.logit(phi[2]))
816
817   log_lik + log_prior + deriv_adjust
818 }
```


The new function calculates the log-derivative of σ with respect to ϕ

```
47 my_logpost <- function(theta, my_info)
48 {
49   # the unknown mean is the first parameter
50   lik_mu <- theta[1]
51   # the unknown standard deviation is the second
52   lik_sigma <- theta[2]
53
54   # log-likelihood -> sum up the independent
55   # log-likelihoods
56   log_lik <- sum(dnorm(x = my_info$xobs,
57                        mean = lik_mu,
58                        sd = lik_sigma,
59                        log = TRUE))
60
61   # the log-prior -> sum up the independent priors
62   log_prior <- dnorm(x = lik_mu,
63                     mean = my_info$mu_0,
64                     sd = my_info$tau_0,
65                     log = TRUE) +
66     dunif(x = lik_sigma,
67          min = my_info$sigma_lwr,
68          max = my_info$sigma_upr,
69          log = TRUE)
70
71   # add the log-likelihood and log-prior
72   log_lik + log_prior
73 }
```

```
785 my_logpost_cv <- function(phi, my_info)
786 {
787   # the unknown mean is the first parameter
788   lik_mu <- phi[1]
789   # the unknown logit-transformed standard deviation
790   # is the second, back transform to sigma
791   lik_sigma <- my_info$sigma_lwr +
792     (my_info$sigma_upr - my_info$sigma_lwr) * boot::inv.logit(phi[2])
793
794   # log-likelihood -> sum up the independent
795   # log-likelihoods
796   log_lik <- sum(dnorm(x = my_info$xobs,
797                        mean = lik_mu,
798                        sd = lik_sigma,
799                        log = TRUE))
800
801   # the log-prior -> sum up the independent priors
802   log_prior <- dnorm(x = lik_mu,
803                     mean = my_info$mu_0,
804                     sd = my_info$tau_0,
805                     log = TRUE) +
806     dunif(x = lik_sigma,
807          min = my_info$sigma_lwr,
808          max = my_info$sigma_upr,
809          log = TRUE)
810
811   # add the log-likelihood and log-prior and account
812   # for the derivative adjustment
813   deriv_adjust <- log(my_info$sigma_upr - my_info$sigma_lwr) +
814     log(boot::inv.logit(phi[2])) +
815     log(1 - boot::inv.logit(phi[2]))
816
817   log_lik + log_prior + deriv_adjust
818 }
```